

Ölçeklenebilir Orta Büyüklükte Ağların Uygulanması

- VLAN Çeşitleri
- Voice Vlan Oluşturma
- Dinamik Trunk Protokolü (DTP)
- Vlan Trunk Protokolü (VTP) ve Yapılandırması
- Vlan Güvenliğini Uygulama
- Spanning Tree Protocol (STP)
- Etherchannel
- Default Gateway Redundancy ve HSRP Yapılandırması

VLAN Çeşitleri

Günümüz ağlarında aynı türde birçok VLAN vardır, bazı VLAN türleri trafik sınıflarına göre ayrılır.

1. Veri VLANI
2. Varsayılan VLAN
3. Native VLAN
4. Yönetim VLAN
5. Ses VLANI

Veri VLANI (Data VLAN)

Kullanıcıların **veri trafiğini** (internet, dosya, uygulama trafiği) taşıyan VLAN türüdür.

- En yaygın kullanılan VLAN türüdür.
- Çalışan bilgisayarları, yazıcıları veya istemcileri birbirinden mantıksal olarak ayırmak için kullanılır.
- Örneğin:
 - Muhasebe birimi VLAN 10
 - İnsan kaynakları VLAN 20
 - Ar-Ge VLAN 30

Amaç: Trafiği bölümlere ayırarak **güvenliği**, **performansı** ve **yönetilebilirliği** artırmak.

Varsayılan VLAN (Default VLAN)

Switch üzerinde ilk tanımlı gelen VLAN'dır. Cisco switchlerde bu VLAN 1'dir.

- Switch ilk açıldığında **tüm portlar VLAN 1'e** aittir.
- Yönetim trafiği (CDP, STP, VTP vb.) varsayılan olarak VLAN 1 üzerinden gider.
- Güvenlik açısından genelde VLAN 1 **aktif olarak kullanılmaz**, sadece teknik altyapı için tutulur.

Amaç: Cihazların ilk yapılandırmasında temel iletişim sağlamak.

Native VLAN

Trunk portlar üzerinden **etkilenmeden (untagged)** geçen trafiğin ait olduğu VLAN'dır.

- Trunk bağlantılarda, VLAN bilgisi genelde **etiket (tag)** ile taşınır (IEEE 802.1Q standardı).
- Ancak bir VLAN, **etiketsiz** trafik alabilir – işte bu VLAN “**Native VLAN**”dır.
- Cisco cihazlarda varsayılan native VLAN genellikle **VLAN 1**'dir, ancak güvenlik için değiştirilir (örneğin VLAN 99 yapılır).

Amaç: Eski cihazlarla uyumluluk sağlamak ve etiketlenmemiş trafiği doğru VLAN'a yönlendirmek.

Yönetim VLAN (Management VLAN)

Switch veya router gibi ağ cihazlarının uzaktan yönetimi (**SSH, Telnet, SNMP, HTTP**) için kullanılan VLAN'dır.

- Genellikle bu VLAN'a **IP adresi atanır**.
- Yöneticiler cihazlara bu IP üzerinden bağlanır.
- **Veri trafiğinden ayrı** tutulur, güvenliği artırır.
- Örnek : VLAN 99 = Management VLAN

Amaç: Ağ yöneticisinin, kullanıcı trafiğine karışmadan cihazları yönetebilmesidir.

Ses VLAN (Voice VLAN)

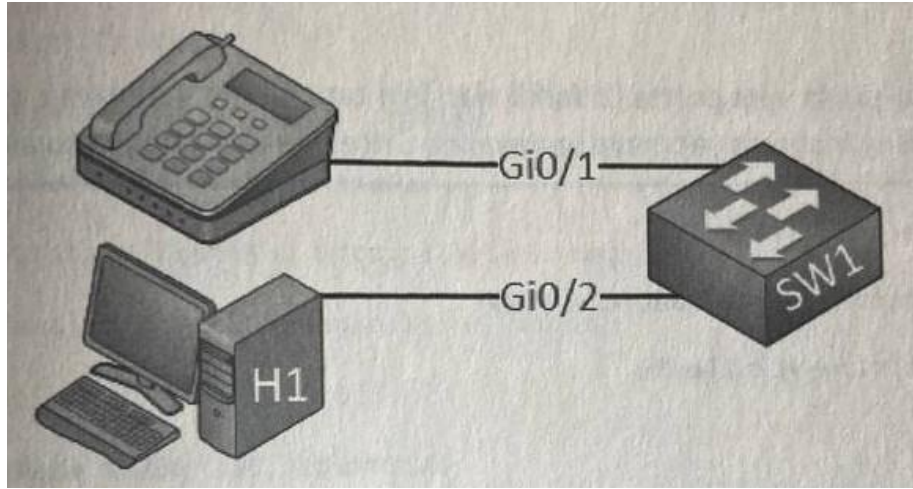
IP telefonların ses (VoIP) trafiğini taşıyan VLAN türüdür.

- IP telefonlar genellikle aynı kablo üzerinden hem bilgisayar hem ses trafiğini taşır.
- Bu yüzden switch portu hem data VLAN hem voice VLAN taşıyabilir.
- Ses trafiğine öncelik verilir (QoS – Quality of Service).

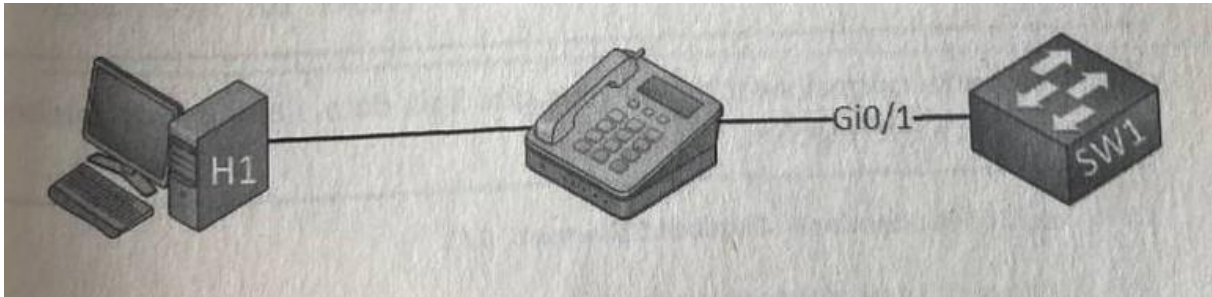
Amaç: Ses iletişiminin gecikmesini, kaliteli ve güvenli olmasını sağlamak.

Voice Vlan (Ses Vlanı)

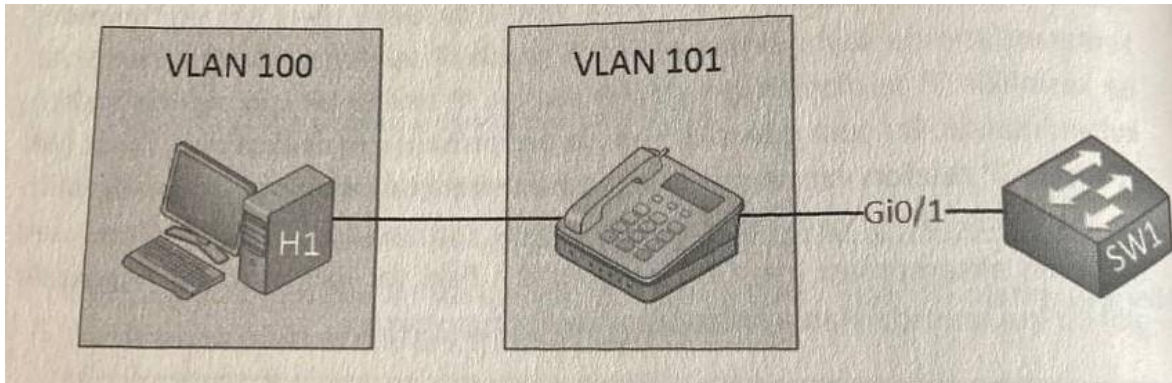
Günümüz ağlarının vazgeçilmez bir parçası da IP telefonlardır. Voice vlan'a bazı kaynaklarda Auxiliary Vlan (AUX VLAN) da denir. Eğer çok sayıda IP telefon ya da IP kamera var ise kesinlikle IP telefonlar için ayrı bir switch, IP kameralar için ayrı bir switch kullanılmalıdır. Bu hem güvenlik hem de performans için akıllıca olur. Fakat çok az sayıda IP telefon var ise mecburen data switch'ine bağlanmaları gerekebilir. Bu durumda da yoğun data trafiği olması durumunda IP telefonların ses trafiğinin aksamaması için QoS yapılmalıdır. Peki ya vlan ayarları? Aşağıdaki gibi bir kullanımda vlan ayarları açısından sorun yoktur:



Burada Gi0/1 portunu telefonlar için kullandığımız vlan'a, Gi0/2 portunu da data vlan'a üye yaparız ve sorun kalmaz. Peki ya aşağıdaki gibi bir durum varsa? Aynı portu hem bilgisayar hem de IP telefon kullanmak zorundaydı?



Bu durumda Gi0/1 portunu özel bir konfigürasyonla ayarlamamız gerekir. Şöyle bir yapımız olduğunu varsayarak bu duruma göre konfigürasyon yapalım:



Bu durumda aynı portta iki farklı vlan için tanımlama yapmamız gerekecektir. Şimdi ayarlarımızı yapmaya başlayalım. Öncelikle vlan'ları oluşturalım:

```
SW1(config)#vlan 100
SW1(config-vlan)#name COMPUTER
SW1(config-vlan)#exit

SW1(config)#vlan 101
SW1(config-vlan)#name VOIP
SW1(config-vlan)#exit
```

Şimdi de portu normal switch bağlıymış gibi ilgili data vlan'ına ve IP telefonu da voice vlan'a üye yapalım.

```
SW1(config)#interface GigabitEthernet 0/1
SW1(config-if)#switchport mode access
SW1(config-if)#switchport access vlan 100
SW1(config-if)#switchport voice vlan 101
SW1(config-if)#exit
```

Şimdi aklınıza şu soru gelebilir. Biz switch portunda ayarları yaptık ama IP telefon kendi vlan'ını nasıl anlayacak ve onu kullanacak? Burada kullanılan IP telefon da Cisco ise switch ile CDP kullanarak haberleşecek ve kendi vlan'ını kolayca öğrenebilecektir. Diğer üreticilerin IP telefonları ise LLDP (Link Layer Discovery Protocol) kullanarak voice vlan bilgisini switch'den öğrenirler.

Son olarak ilgili portta bir de kontrolümüzü yapalım bakalım her şey istediğimiz gibi olmuş mu:

```
SW1#show interfaces GigabitEthernet 0/1 switchport
```

```
Name: Gi0/1
```

```
Switchport: Enabled
```

```
Administrative Mode: static access
```

```
Operational Mode: static access
```

```
Administrative Trunking Encapsulation: negotiate
```

```
Operational Trunking Encapsulation: native
```

```
Negotiation of Trunking: Off
```

```
Access Mode VLAN: 100 (COMPUTER)
```

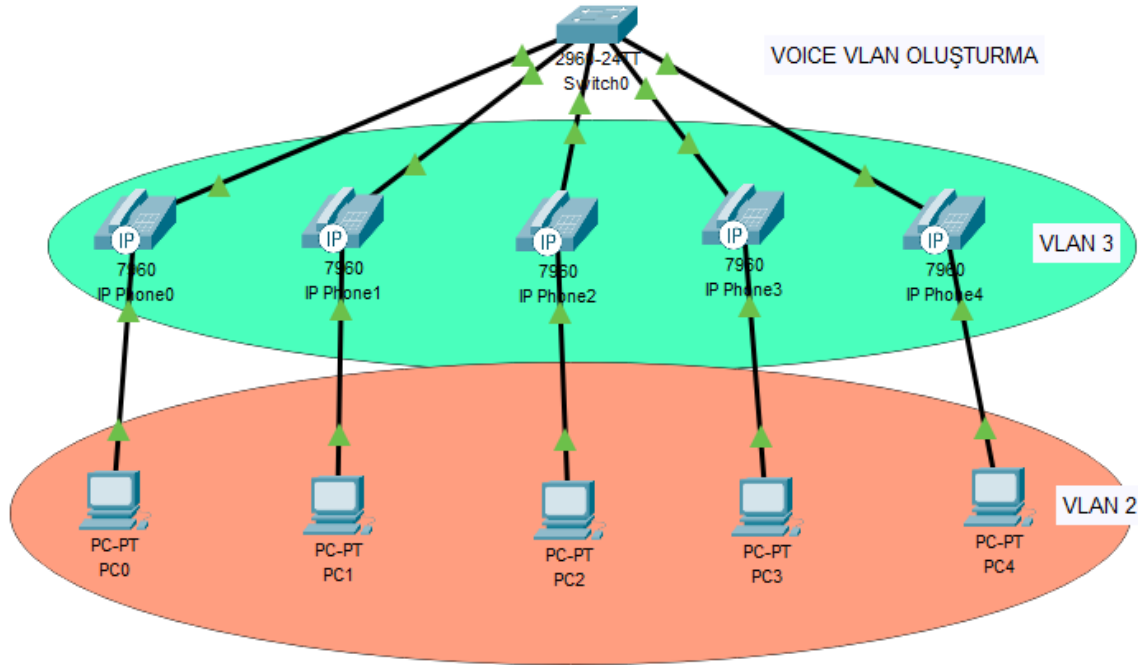
```
Trunking Native Mode VLAN: 1 (default)
```

```
Administrative Native VLAN tagging: enabled
```

```
Voice VLAN: 101 (VOIP)
```

Packet Tracer Uygulaması – Voice Vlan Oluşturma

Örnekte IP telefon yapılandırması değil de sadece ses için nasıl vlan oluşturuluyor, o gösterilmiştir. Öncelikle aşağıdaki gibi bir yapımız var:



Telefonlar düz kablo ile switch portundan switch'e, PC portunda ise bilgisayarlara bağlıdır. Fakat ilk takıldığında kabloda yeşil ibaresini görmeyiz çünkü power kablusunun takılması gerekir. The Cisco VoIP power adapter:



Not: POE desteği olursa gücü portlar üzerinden de alabilir. Böylelikle yukarıdaki gibi bir adaptöre gerek kalmaz.

Örneğimize göre diyelim ki 24 adet IP telefonumuz ve 24 adet PC'miz var. Bu sayede 24 port'luk bir switch işimizi görür. Diğer türlü 48 port gerekecekti. Aynı port üzerinden 2 vlan oluşturacağız.

Öncelikle vlan'larımızı oluşturalım:

```
Switch(config)#  
Switch(config)#vlan 2  
Switch(config-vlan)#name veri  
Switch(config-vlan)#vlan 3  
Switch(config-vlan)#name telefon  
Switch(config-vlan)#end
```

Show vlan brief komutu ile oluşturduğumuz bu vlan'ları görelim:

2	veri	active	Gi0/17, Gi0/22
3	telefon	active	

Örneğimizde 5'er adet telefon ve PC var fakat biz 24 adet olduğunu düşünelim ve bu vlan'ları switchimizin tüm portları için atayalım. Bunun için tüm portlarımızın interface'ine giriyoruz.

```
Switch(config)#interface range fastEthernet 0/1-24
Switch(config-if-range)#switchport access vlan 2
Switch(config-if-range)#switchport voice vlan 3
Switch(config-if-range)#end
```

Tekrar sh vl br diyelim.

2	veri	active	Fa0/1, Fa0/2, Fa0/3, Fa0/4 Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Fa0/24
3	telefon	active	Fa0/1, Fa0/2, Fa0/3, Fa0/4 Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Fa0/24

Kontrol edelim:

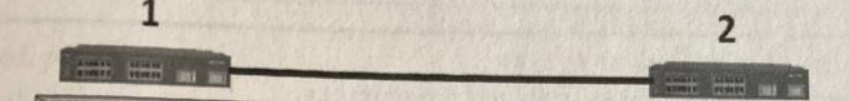
```
Switch#show interfaces fastEthernet 0/1 switchport
Name: Fa0/1
Switchport: Enabled
Administrative Mode: dynamic auto
Operational Mode: static access
Administrative Trunking Encapsulation: dot1q
Operational Trunking Encapsulation: native
Negotiation of Trunking: On
Access Mode VLAN: 2 (veri)
Trunking Native Mode VLAN: 1 (default)
Voice VLAN: 3
Administrative private-vlan host-association: none
Administrative private-vlan mapping: none
Administrative private-vlan trunk native VLAN: none
Administrative private-vlan trunk encapsulation: dot1q
Administrative private-vlan trunk normal VLANs: none
Administrative private-vlan trunk private VLANs: none
Operational private-vlan: none
Trunking VLANs Enabled: All
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
Capture VLANs Allowed: ALL
Protected: false
```

Dynamic Trunking Protocol (DTP)

Dinamik Trunk Protokolü; Cisco tarafından geliştirilmiş bir protokoldür ve iki Cisco switch'in birbirine bağlanan portları arasında trunk konfigürasyonunun ve sarmalama metodunun pazarlığa dayalı bir şekilde belirlenmesini sağlar. Dinamik Trunk Protokolü sayesinde portlar arası trunk olma işlemi otomatik olarak yapılabilir. Bu protokol OSI referans modeline göre 2. Katmanda çalışır. Dinamik Trunk Protokolü'nün gerçekleştirilmesi için uygulanabilecek 4 çeşit mod vardır. Bunlar; **Dynamic Desirable**, **Dynamic Auto**, **Trunk** ve **Access**'tir.

Bu modlardan Dynamic Desirable ve Dynamic Auto portun otomatik olarak trunk'a geçip geçmeyeceğini belirler ve hat boyunca DTP paketlerinin gönderilmesiyle ilgilidir. Bu mod'lar trunk olmaya meyllidir. "**show interfaces trunk**" komutu kullanılarak bu görülebilir.

Desirable modunda aktif olarak 30 saniyede bir DTP paketi gönderilir. Auto ise pasif bir moddur ve bu mod etkinken sadece karşı cihazdan gelen DTP paketlerine cevap verilir. Trunk ve Access Modu ise ağ yöneticisinin portun manuel olarak Trunk ya da non-Trunk (Access) olarak belirlemesine olanak sağlar. Aşağıdaki tabloda hangi karşılıklı mod yapılandırmalarının başarılı bir DTP işlemini gerçekleştireceği gösterilmiştir.



SWITCH 2 \ SWITCH 1	Access ve No-negotiate	Access	Dynamic Auto	Dynamic Desirable	Trunk	Trunk ve No-negotiate
Access ve No-negotiate	ACCESS	ACCESS	ACCESS	ACCESS	X	X
Access	ACCESS	ACCESS	ACCESS	ACCESS	X	X
Dynamic Auto	ACCESS	ACCESS	ACCESS	TRUNK	TRUNK	X
Dynamic Desirable	ACCESS	ACCESS	TRUNK	TRUNK	TRUNK	X
Trunk	X	X	TRUNK	TRUNK	TRUNK	TRUNK
Trunk ve No-negotiate	X	X	X	X	TRUNK	TRUNK

X Beklenmeyen sonuçların olduğu durumları göstermektedir.

Özetle, DTP (Dynamic Trunking Protocol), Cisco switch'lerde portlar arasında trunk bağlantısını **otomatik olarak pazarlayıp oluşturmak için kullanılan** bir protokoldür. Switch portlarının trunk mı yoksa access mi olacağını belirler; **dynamic auto** pasif olarak trunk teklifini beklerken, **dynamic desirable** aktif olarak trunk oluşturmak ister. Sadece Cisco cihazlarda bulunur ve güvenlik nedeniyle günümüzde çoğunlukla **manuel trunk yapılandırması** tercih edilir.

DTP; VTP (VLAN Trunking Protocol) ile karıştırılmamalıdır. Çünkü **VTP trunk olarak birbirine bağlanmış ve aynı VTP domaininde bulunan iki switch'in VLAN bilgisini paylaşmasını sağlamaya yönelik olarak geliştirilmiştir. DTP ise iki switch'in birbirine bağlanan portlarının paket trafiğinin analizi sonucu otomatik trunk yapılmasını belirleyen protokoldür.** Ancak dinamik trunk belirlenmesi için işlemin yapılacağı karşılıklı interface'ler aynı VTP domainin de bulunmalıdır.

Aşağıdaki komut DTP durumunu gözlemlememizi sağlar. Buna göre 30 saniyede bir DTP hello paketi gönderilir.

```
Switch#show dtp
Global DTP information
  Sending DTP Hello packets every 30 seconds
  Dynamic Trunk timeout is 300 seconds
  2 interfaces using DTP
```

Varsayılan olarak switch'ler "Dynamic Desirable" mod'dadır. Ancak bu sürekli DTP paketleri gönderilmesine sebep olacağından fazladan bir yük bindirecek ve portların her an trunk mod'a geçip vlan bilgisi gibi bazı bilgileri bu port üzerinden akıtma ihtimalini bulundurduğu için bir güvenlik açığı oluşturacaktır. Bu istenmeyen bir durumdur. Bu nedenle DTP kullanılmayacak portların bu özelliğinin devre dışı bırakılmasını sağlamak daha mantıklı olacaktır. Bu işlem aşağıdaki komutla yapılmaktadır:

```
SW1(config)#int ran fa 0/1-24
SW1(config-if-range)#switchport nonegotiate

Command rejected: Conflict between 'nonegotiate' and 'dynamic'
status.
```

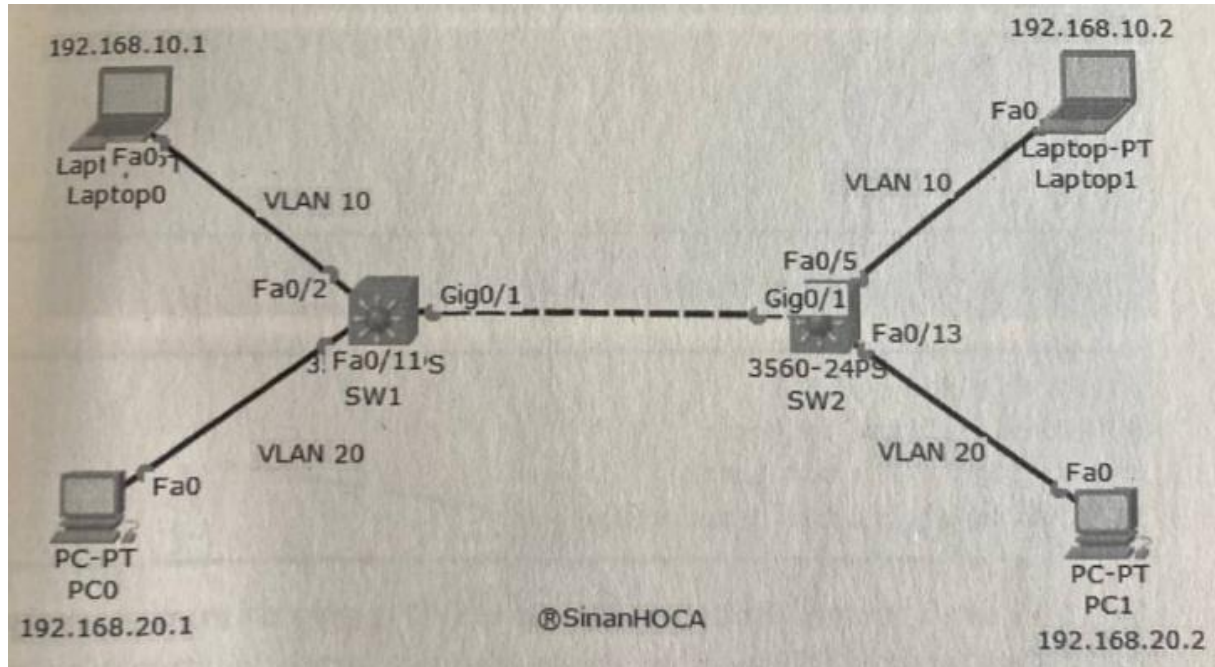
Yukarıdaki hata portun "Dynamic" modda olması durumunda ortaya çıkar. Bu durumu düzeltmek için;

```
SW1(config)#int ran fa 0/1-24
SW1(config-if-range)#sw mod access
SW1(config-if-range)#sw nonegotiate
```

Şeklinde önce port dinamik mod'dan çıkartılır, sonra negotiate devre dışı bırakılır. Aynı işlem port "Access" mod'a alınarak da yapılabilir.

Belli bir portun konfigürasyonuna bakıldığında DTP'nin etkin olup olmadığını ve etkinse hangi modunun etkin olduğunu görebiliriz. Switchport nonegotiate komutu yazılmadığı sürece portlar "Trunk" veya "Access" modunda seçilse dahi portlar arasında DTP paketleri gönderilir.

DTP Uygulaması



Önce switch'lerde temel vlan ayarlarını yapalım:

```
en
conf t
host SW1
vlan 10
vlan 20
int ran fa 0/1-10
sw mod acc
sw acc vlan 10
exit
int ran fa 0/11-20
sw mod acc
sw acc vlan 20
exit
en

conf t
host SW2
vlan 10
vlan 20
int ran fa 0/1-10
sw mod acc
sw acc vlan 10
exit
int ran fa 0/11-20
sw mod acc
sw acc vlan 20
exit
```

Şimdi de **gi 0/1** portunun **trunk** ayarlarını yapalım.

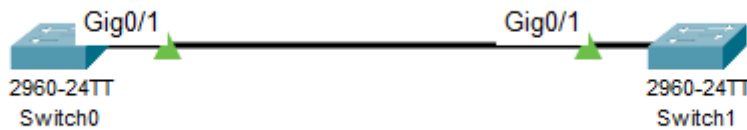
```
SW1(config)#int gi 0/1
SW1(config-if)#sw tr enc d
SW1(config-if)#sw mod trunk
SW1(config-if)#sw mod dynamic desirable
```

Evet SW1 tarafı tamam. Şimdi diyeceksiniz ki SW2 tarafını da trunk yapmalıyız. Hayır! Karşı taraftaki DTP ayarı sayesinde karşı taraf ayarlarını otomatik olarak yapacaktır. Bilgisayarlara da IP verip denediğinizde karşılıklı ping atabildiğini görürsünüz.

DTP – Packet Tracer Uygulaması

	Dynamic Auto	Dynamic Desirable	Trunk	Access
Dynamic Auto	Access	Trunk	Trunk	Access
Dynamic Desirable	Trunk	Trunk	Trunk	Access
Trunk	Trunk	Trunk	Trunk	Limited connectivity
Access	Access	Access	Limited connectivity	Access

Aşağıdaki gibi 2 adet switch gi 0/1 portlarından birbirine bağlıdır: Henüz vlan yapılandırması yok.



Öncelikle bakalım bir trunk'ımız var mı. Soldaki ve sağdaki switch için baktığımızda herhangi bir trunk yok.

```
Switch#sh int tr
```

```
Switch#
```

```
Switch#sh int tr
```

```
Switch#
```

Şuan iki switch de dynamic auto modunda: Tabloya göre ikisi de auto modunda ise acces ile anlaşıyorlar. Dynamic Auto demek; karşı taraf trunk isterse kabul eder yani pasif olarak bekler. Şuan ikisi de pasiftir ve trunk söz konusu olamaz.

```
Switch#show interfaces switchport
Name: Fa0/1
Switchport: Enabled
Administrative Mode: dynamic auto
```

Bu arada iki switch arasında DTP mesajları gidip gelir (30 saniyede bir). Auto durumunda trunk teklifi yapılmaz, eğer teklif gelirse kabul eder. Bu mesajlar tabi bant genişliğini işgal ediyor.

NOT: **Negotiation** (müzakere) ağlarda iki cihazın bağlantı ile ilgili bazı özellikleri otomatik olarak anlaşması demektir.

Negotiation'ı kapattığımızda bu mesajlar gidip gelmeyecek artık.

Öncelikle şunu yapalım; soldaki switch'imizi manuel olarak trunk moda çekelim. Aşağıda hatırlayacağımız üzere switch portuna host bağlıyorsak access, eğer switch'i switch'e bağlıyorsak trunk yapıyorduk. Dynamic auto da zaten varsayılan moddu. Biz aşağıda manuel olarak trunk moda çektik (layer3 switch'lerde direkt böyle çekilmiyor, ileride gösterilecek).

```
Switch(config)#interface gigabitEthernet 0/1
Switch(config-if)#switchport mode ?
    access    Set trunking mode to ACCESS unconditionally
    dynamic   Set trunking mode to dynamically negotiate access or trunk mode
    trunk     Set trunking mode to TRUNK unconditionally
Switch(config-if)#switchport mode dynamic ?
    auto      Set trunking mode dynamic negotiation parameter to AUTO
    desirable Set trunking mode dynamic negotiation parameter to DESIRABLE
Switch(config-if)#switchport mode trunk

Switch(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to down
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up
```

Aşağıdaki gibi değişti:

```
Switch#show interfaces gigabitEthernet 0/1 switchport
Name: Gig0/1
Switchport: Enabled
Administrative Mode: trunk
Operational Mode: trunk
Administrative Trunking Encapsulation: dot1q
```

Tahmin edileceği üzere karşı taraf da baktı ki karşı taraf trunk, pasif olduğu için o da otomatik olarak trunk moda geçti.

```
Switch#show interfaces gigabitEthernet 0/1 switchport
Name: Gig0/1
Switchport: Enabled
Administrative Mode: dynamic auto
Operational Mode: trunk
```

Ama cisco'nun önerdiğini bunu auto olarak bırakmadan sağdaki switch'in de manuel olarak trunk moda çekilmesi.

```
Switch(config)#interface gigabitEthernet 0/1
Switch(config-if)#switchport mode trunk
Switch(config-if)#
```

Görüldüğü gibi auto'dan trunk'a geçti:

```
Switch#show interfaces gigabitEthernet 0/1 switchport
Name: Gig0/1
Switchport: Enabled
Administrative Mode: trunk
Operational Mode: trunk
```

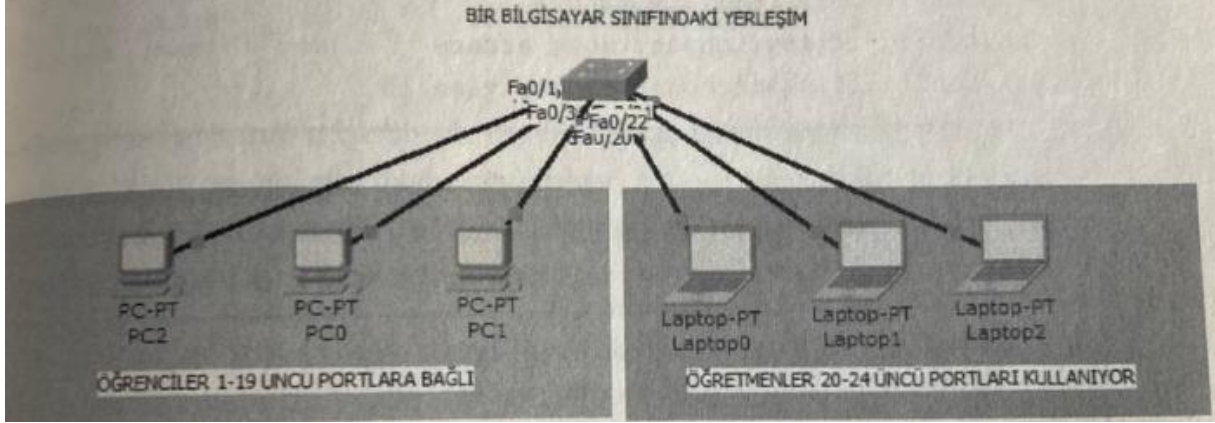
İki tarafı da yaptık, artık bizim negotiation'a ihtiyacımız yok, onu kapatalım. Çünkü bu otomatik anlaşmalarıydı, fakat biz kendimiz yaptık zaten. Aşağıdaki gibi 2 switch için de ilgili port arayüzüne girip **nonegotiate** diyerek otomatik müzakere isteğini kapatıyoruz.

```
Switch(config)#interface gigabitEthernet 0/1
Switch(config-if)#switchport nonegotiate
Switch(config-if)#
```

Artık birbirlerine DTP mesajları göndermeyecekler.

VLAN Yapısına Giriş – VTP Domain

Neden Vlan'lara ihtiyacımız var?



Yukarıdaki şekildeki gibi bir okul düşünün. Herhangi bir özel tanımlama yapılmadığı takdirde tüm kullanıcılar, diğer tüm kullanıcılara erişebilmektedir. Hatta eğer ağda kablosuz erişim de varsa sormayın gitsin, gelen misafirler de tüm ağa erişebilir haldedir. Herkesin birbirine erişmesi iyi gibi görünse de istenen bir durum olmayabilir. Çünkü temelde tüm kullanıcıların erişmek istedikleri yerler bellidir. Herkes öncelikle internet, sonrasında da kurum sunucularına erişimi ister. Bunun dışındaki erişimler genelde gereksiz ve sorun çıkarabilecek türden olabilir. Çünkü aynı broadcast domain'in de ne kadar çok bilgisayar varsa o kadar ağda broadcast ve yavaşlamalar olabilecek, ağdaki bir bilgisayarı ele geçiren bir hacker ya da Trojan, worm, virüs bulunduğu tüm ağa ulaşabilecektir. Bu bakış açısıyla networkü küçük parçalara bölmek çok doğru bir harekettir.

Şimdi yukardaki örnek ile başlayarak vlan'lar oluşturup network'ü küçük parçalara (subnet) bölelim.

Öncelikle switch'te vlan'lar oluşturalım. Vlan'ları oluşturmadan önce bu bilgisayarların hepsinin aynı IP grubunda olduğunu ve birbirine ulaşabildiğini görebilirsiniz.

```
Switch(config)#vlan 10
Switch(config-vlan)#name ogrenciler
Switch(config)#vlan 20
Switch(config-vlan)#name ogretmenler
```

Şimdi switch'in hangi portunun, hangi vlan'da olacağını belirleyelim. Bunun için;

```
Switch(config-if)#interface fa 0/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 10
```

Yukarıdaki komutlar ile fa 0/1 interface'indeki bilgisayarı 10. Vlan'a almış olduk. Fakat 19 portu bu vlan'a almak uğraştırır. Şimdi belli bir aralıktaki tüm portlara aynı anda işlem yapmayı sağlayan range komutunu kullanalım:

```
Switch(config)#interface range fa 0/2-19
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 10
```

Tamam, şimdi de öğretmen vlan'ının tanımlarını yapalım:

```
Switch(config)#interface range fa 0/20-24
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 20
```

Şimdi oluşturduğumuz vlan'ları ve bu vlan'lara üye olan portları kontrol edelim.

Komutumuz show vlan:

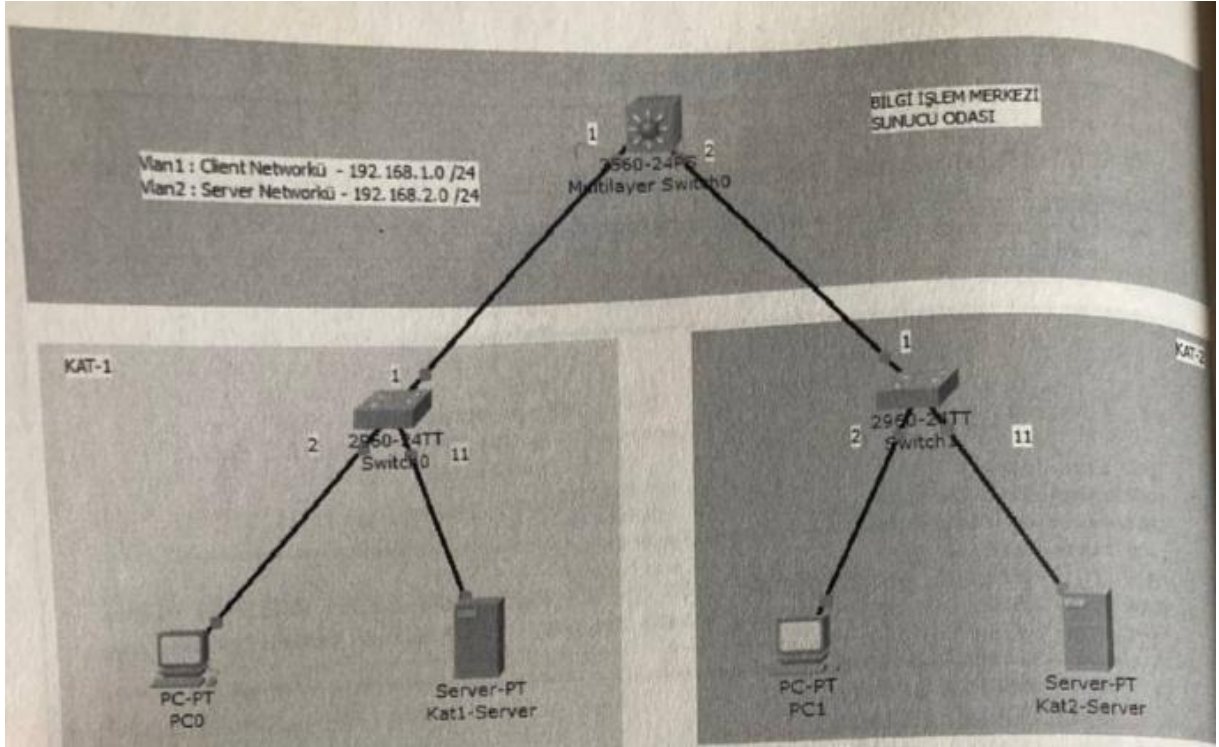
```
Switch#
Switch#
Switch#sh vl
```

VLAN Name	Status	Ports
1 default	active	Gig1/1, Gig1/2
10 ogrenciler	active	Fa0/1, Fa0/2, Fa0/3, Fa0/4 Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19 Fa0/20, Fa0/21, Fa0/22, Fa0/23 Fa0/24
20 ogretmenler	active	
1002 fddi-default	act/unsup	
1003 token-ring-default	act/unsup	
1004 fddinet-default	act/unsup	
1005 trnet-default	act/unsup	

VLAN	Type	SAID	MTU	Parent	RingNo	BridgeNo	Stp	BrdgMode	Trans1	Trans2
1	enet	100001	1500	-	-	-	-	-	0	0
10	enet	100010	1500	-	-	-	-	-	0	0
20	enet	100020	1500	-	-	-	-	-	0	0
1002	fddi	101002	1500	-	-	-	-	-	0	0

Bu aşamadan sonra artık bir vlan'daki bilgisayarların IP'si ne olursa olsun diğer vlan'a ulaşması mümkün olmayacaktır. Eğer diğer vlan'lara da ulaşmasını istersek o zaman vlan'ları L3 bir switch ya da router'de sonlandırıp, IP verip vlan'lar arası geçişleri kontrol edebiliriz.

Şimdi olayı bir boyut ileriye taşıyalım ve daha karmaşık bir yerdeki vlan yapısını kuralım. Çok katlı bir yerde her katta farklı bölümlerin görev yaptığını ve bu bölümlerin birbirine erişimlerinin kontrol altında tutulduğu bir yapıyı kuralım.



Çok sayıda switch'in olduğu ve her switch'te bir sürü vlan'ın olduğu bir ağda, switch'lerde tek tek tüm vlan'ları oluşturmak ciddi zaman alacaktır. Bu yüzden merkezi bir switch'i VTP server switch olarak belirleyip, diğer switch'lerin bu switch'den tüm vlan'ları otomatik olarak çekmesini sağlayabiliriz.

Öncelikle oluşturacağımız ve dağıtımını yapacağımız VTP domain'ine bir isim veriyoruz:

```
Switch(config)#vtp domain SinanHOCA
```

Sonrasında ilgili switch'in VTP modu'na karar veriyoruz. Tüm switch'ler default da, server mod'da gelirler. Server mode üzerindeki vlan'ları client mod'da olan switch'lere aktarır.

```
Switch(config)#vtp mode server
```

Şimdi artık L3 switch'imiz de vlan'ları oluşturalım. En üstteki bilgi işlem odamızdaki switch'te vlan'ları oluşturalım. Default da tüm switch'lerde vlan1 vardır ve tüm portlar bu vlan'a üyedir. Vlan'ları görmek için user mod'da:

```
Switch#sh vlan
```

Artık üstteki L3 switch'de vlan'ları oluşturmaya başlayabiliriz.

```
Switch(config)#vlan 2
Switch(config-vlan)#name client

Switch(config)#vlan 3
Switch(config-vlan)#name server

Switch(config)#vlan 99
Switch(config-vlan)#name management
```

Şimdi vlan'lara IP verelim. Vlan'ları sanki birer sanal port, sanal interface gibi düşünüp aynı fastEthernet portlara verdiğimiz gibi her vlan'a IP verelim.

```
Switch(config)#interface vlan 2
Switch(config-if)#ip address 192.168.2.1 255.255.255.0
>>>> Genelde karışmasın düzenli olsun diye her vlandaki IP yapısı o
vlanın ismine göre düzenlenir.

Switch(config-if)#interface vlan 3
Switch(config-if)#ip address 192.168.3.1 255.255.255.0

Switch(config-if)#interface vlan 99
Switch(config-if)#ip address 192.168.99.1 255.255.255.0
Switch(config-if)#description bu vlan yönetim icindir.
>>>> İstersek burada olduğu gibi her vlan interface'ine bir
açıklama da girebiliriz.
```

ÖNEMLİ: Vlan'lara verdiğimiz IP'ler o vlan'daki client'lerin **Default Gateway** yani **Ağ Geçidi** olacaktır. Böylece o network'tan farklı network'lara gitmek isteyen paketler bu L3 switch'e gelecek ve yönlendirme ile istenilen network'e ulaştırılabilecektir. "Madem diğer network'larla görüştürecektik neden eziyet çekip ağıımızı vlan'lara ayırdık?" sorusunun cevabı için; vlan'lar broadcast domain'lerini küçülttüklerinden ağ hızını arttırırlar ve diğer vlan ya da network'lara geçişler de ACL ile kontrol edilebilir. Günümüz teknolojisinde gelişmiş UTM cihazlarında bile bu vlan'lar

sonlandırabilmekte ve vlan'lar arası geçişin kontrol edildiği gibi bu geçişte saldırı, virüs kontrolleri bile yapılabilir.

Şimdi sıra geldi L3 switchin kendi üzerindeki tanımladığımız bu vlan'ları diğer switch'lere aktarmasına. Bunu aradaki bağlantıyı **TRUNK** yaparak sağlıyoruz. L3 switch de fa 0/1 ve fa 0/2 interface'leri için trunk port tanımlayalım.

```
sistemodasi(config)#interface fa 0/1
sistemodasi(config-if)#switchport trunk encapsulation dot1q
sistemodasi(config-if)#switchport mode trunk

sistemodasi(config)#interface fa 0/2
sistemodasi(config-if)#switchport trunk encapsulation dot1q
sistemodasi(config-if)#switchport mode trunk
```

Kat switch'lerinde ise L2 switch'teki trunk komutlarını kullanıyoruz. Fa 0/1 portlarını trunk yapalım:

```
kat-1(config)#interface fa 0/1
kat-1(config-if)#switchport mode trunk

kat-2(config)#interface fa 0/1
kat-2(config-if)#switchport mode trunk
```

Şimdi de bu kat switch'lerine VTP client olduklarını söyleyelim ki bizim omurga switch'ten vlan bilgilerini çeksinsinler. Hadi bakalım:

```
kat-1(config)#vtp mode client
kat-1(config)#vtp domain SinanHOCA

kat-2(config)#vtp mode client
kat-2(config)#vtp domain SinanHOCA
```

Eğer önüne gelen switch takıp bizim sistemimize entegre olmasın ulaşamasın diyorsak o zaman VTP domain'ine şifre vermeli ve bunu tüm VTP switch'lerde yapmalıyız:

```
#vtp password Ankara2013
```

Şimdi vlan'larımız gelmiş mi bir kontrol edelim:

```
Switch(config)#do sh vl
```

VLAN	Name	Status	Ports
1	default	active	Fa0/2, Fa0/3, Fa0/4, Fa0/5 Fa0/6, Fa0/7, Fa0/8, Fa0/9 Fa0/10, Fa0/11, Fa0/12, Fa0/13 Fa0/14, Fa0/15, Fa0/16, Fa0/17 Fa0/18, Fa0/19, Fa0/20, Fa0/21 Fa0/22, Fa0/23, Fa0/24, Gig1/1 Gig1/2
2	client	active	
3	server	active	
99	management	active	
1002	fddi-default	act/unsup	
1003	token-ring-default	act/unsup	
1004	fddinet-default	act/unsup	
1005	trnet-default	act/unsup	

Evet vlan'larımızın geldiğini görüyoruz. "Peki vlan 99 management vlan'ı ne?" Biz network yöneticileri oturduğumuz yerden tüm ağ cihazlarına ulaşabiliyor olmalıyız ki onları yönetebilelim, oluşan sorunlara anında müdahale edelim. Bir cihaza uzaktan erişmek için onun IP'si olmalıdır. Peki o zaman tüm cihazlara IP vereceksek hangi IP'yi verelim? İşte bu noktada management vlan kullanılır. Bir vlan (IP bloğu) belirleriz ve tüm cihazlara bu vlan'dan IP'ler veririz. Böylece gerek ağ yönetimi yazılımları bu aynı bloktaki cihazlara erişebilir, hem de biz tüm cihazlarımızı rahatlıkla yönetebiliriz. Management vlan kullanmanın bir yararı da bu farklı yönetim ağına geçişleri istersek router'dan sınırlandırabilir, bu vlan'a yalnızca şu şu IP'ler ulaşabilsin diyebiliriz. Ağ cihazlarına farklı ağlardan da erişmek için bu cihazların yalnızca IP ve Subnet Mask'lerinin olması yeterli olmaz ve ağ geçidi de tanımlanır. İşte bunun için oluşturduğumuz ve IP verdiğimiz bu yönetim vlan'ını diğer switch'lerde de default gateway olarak tanımlıyoruz.

```
kat-1(config)#ip default-gateway 192.168.99.1  
kat-2(config)#ip default-gateway 192.168.99.1
```

Evet şimdi de artık kat switch'lerinin portlarındaki bağlantıları tanımlamamız gerekiyor. Hangi port hangi vlan'da olacak?

```
kat-1(config)#interface fa 0/2
kat-1(config-if)#switchport access vlan 2
kat-1(config-if)#switchport mode access

kat-1(config)#interface fa 0/11
kat-1(config-if)#switchport access vlan 3
kat-1(config-if)#switchport mode access

kat-1(config)#interface fa 0/2
kat-1(config-if)#switchport access vlan 2
kat-1(config-if)#switchport mode access

kat-2(config)#interface fa 0/11
kat-2(config-if)#switchport access vlan 3
kat-2(config-if)#switchport mode access
```

Bu aşamadan sonra farklı kattaki aynı vlan'daki bilgisayarların birbirine ulaştıklarını görürüz. Diğer vlan'lara ulaşması için L3 switch de routing komutunu girmemiz gerekir.

```
#ip routing
```

Eğer trunk porttan yalnızca bazı vlan'ların geçmesini istiyorsak bunu da belirtebiliriz:

```
#switchport trunk allowed vlan 1-10
```

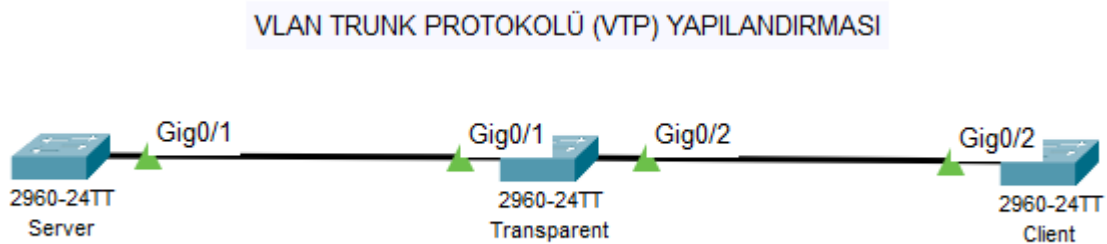
Farklı interface aralıklarında işlem yapılmak istendiğinde:

```
Switch(config)# interface range fastethernet 0/1 - 5
Switch(config)# interface range fastethernet
0/1 - 3, gigabitethernet0/1 - 2
```

NOT: VTP sadece cisco cihazlarda çalışan bir protokoldür.

Packet Tracer – VTP Uygulaması

Aşağıdaki gibi bir yapı var, Server olan Client Switch'lere Vlan bilgilerini trunk ile gönderecek. Ortadakinin Transparent olması bu bilgilerde etkilenmeyecek olması, yani üzerinden geçip gidecek. İstenirse ona da tabi farklı vlan'lar atanabilir.



Her switch'in default'da server modda geldiğini söylemiştik (Şuan bu işlemleri Server Modundaki switch için yapıyoruz).

```
Switch(config)#vtp mode server
Device mode already VTP SERVER.
Switch(config)#
```

O zaman şimdi domain ve parola belirleyelim:

```
Switch(config)#
Switch(config)#vtp domain ccna
Changing VTP domain name from NULL to ccna
Switch(config)#vtp password 12345
Setting device VLAN database password to 12345
Switch(config)#
```

Şimdi Transparent switch'in VTP yapılandırmasına bakalım:

```

Switch#
Switch#show vtp status
VTP Version capable      : 1 to 2
VTP version running      : 1
VTP Domain Name          :
VTP Pruning Mode         : Disabled
VTP Traps Generation     : Disabled
Device ID                : 0000.0C5B.3500
Configuration last modified by 0.0.0.0 at 0-0-00 00:00:00
Local updater ID is 0.0.0.0 (no valid interface found)

Feature VLAN :
-----
VTP Operating Mode      : Server
Maximum VLANs supported locally : 255
Number of existing VLANs : 5
Configuration Revision  : 0
MD5 digest              : 0x7D 0x5A 0xA6 0x0E 0x9A 0x72 0xA0 0x3A
                        : 0xF0 0x58 0x10 0x6C 0x9C 0x0F 0xA0 0xF7
Switch#

```

Tabi başlangıçta server modda, bunu değiştirmemiz gerek.

```

Transparent(config)#vtp mode Tra
Transparent(config)#vtp mode Transparent
Setting device to VTP TRANSPARENT mode.
Transparent(config)#vtp domain ccna
Changing VTP domain name from NULL to ccna
Transparent(config)#vtp password 12345
Setting device VLAN database password to 12345
Transparent(config)#

```

NOT: Parola ve Domain tüm switch'lerde aynı olmak zorunda.

Şimdi de client switch'de yapılandırmamızı yapalım:

```

Client(config)#
Client(config)#vtp mode client
Device mode already VTP CLIENT.
Client(config)#vtp domain ccna
Changing VTP domain name from NULL to ccna
Client(config)#vtp password 12345
Setting device VLAN database password to 12345
Client(config)#

```

Yukarda already client diyor fakat ben onu zaten değiştirmişt看.

Şimdi server switch'te vlan'ları oluşturalım:

```

Switch(config)#
Switch(config)#vlan 10
Switch(config-vlan)#vlan 20
Switch(config-vlan)#vlan 30
Switch(config-vlan)#name HR
Switch(config-vlan)#vlan 40
Switch(config-vlan)#name IT
Switch(config-vlan)#end

```

Oluşturduğumuz vlan'lara bakalım:

```
Switch#sh vl br
```

VLAN	Name	Status	Ports
1	default	active	Fa0/1, Fa0/2, Fa0/3, Fa0/4 Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Fa0/24 Gig0/1, Gig0/2
10	VLAN0010	active	
20	VLAN0020	active	
30	HR	active	
40	IT	active	

Şimdi client switch'lere bu bilgilerin gelmesi lazım. Ama biz aradaki TRUNK yapılandırmasını yapmayı unuttuk. O zaman onu yapalım:

```
Server(config)#int gi
Server(config)#int gigabitEthernet 0/1
Server(config-if)#sw mo tr

Server(config-if)#
```

Daha sonra transparent switch'te yapalım (iki port içinde):

```
Transparent(config)#int range gi
Transparent(config)#int range gigabitEthernet 0/1-2
Transparent(config-if-range)#sw mo tr
```

Son olarak client switch'de de bağlı olduğu portu trunk moda çekelim:

```
Client(config)#interface gigabitEthernet 0/2
Client(config-if)#sw
Client(config-if)#switchport mode trunk
Client(config-if)#
```

Artık client switch'de de vlan bilgilerini görebiliriz:

```
Client#sh vl br
```

VLAN	Name	Status	Ports
1	default	active	Fa0/1, Fa0/2, Fa0/3, Fa0/4 Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Fa0/24 Gig0/1
10	VLAN0010	active	
20	VLAN0020	active	
30	HR	active	
40	IT	active	

Bakalım transparent switch’de herhangi bir vlan bilgisi var mı?

```
Transparent#sh vl br
```

VLAN	Name	Status	Ports
1	default	active	Fa0/1, Fa0/2, Fa0/3, Fa0/4 Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Fa0/24
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddi-default-5	active	

Görüldüğü gibi yok.

NOT: Client moddaki switch VLAN oluşturamaz: fakat transparent switch’de oluşturulabilir.

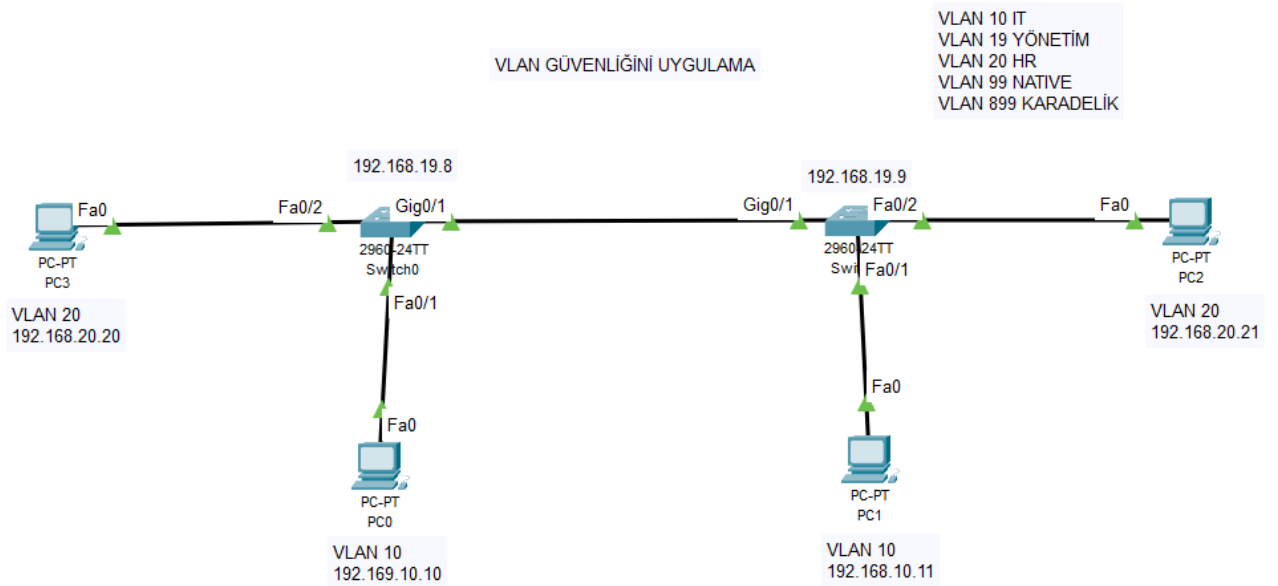
Vlan Güvenliğini Uygulama

Bir önceki yazıda olduğu gibi switch'leri client modda bırakmak tehlikelidir. Cisco'nun önerdiği üzere işimiz bittiğinde transparent moda almamız gerekir. Bunun sebebi VTP (VLAN Trunking Protocol)'da **Configuration Revision** (Yapılandırma Revizyonu) değeridir. Bu değer, bir VTP domain'indeki VLAN veritabanının **güncellik seviyesini** gösteren sayısal bir **sayaçtır**. Bu değer, VTP reklam mesajlarında (summary/Advertisement) taşınır ve hangi switch'in en güncel VLAN bilgisine sahip olduğunu belirler.

Her VLAN değişikliği (ekleme, silme, isim değiştirme, vs.) yapıldığında, VTP Server switch'te revision numarası 1 artar. VTP Client switch'ler, gelen VTP reklamındaki revision numarasını kendi revision numarasıyla karşılaştırır; Eğer server'ın revision numarası kendi revision numarasından büyük ise bir değişiklik yaşandığını algılar ve kendini günceller. Eğer değilse yoksayar.

Burdaki tehlike; eğer yanlışlıkla yüksek revision'lu bir switch'i bağlarsak, ağdaki tüm VLAN'lar silinebilir. Örneğin, yeni bir switch (revision 100) VTP domain'e katılırsa, diğer tüm switch'ler VLAN veri tabanını bu boş switch'ten alır ve tüm VLAN'lar kaybolur. Bununla ilgili Kali Linux altında **yersinia** adında atak saldırıları için bir tool da bulunur.

Şimdi Aşağıdaki gibi bir yapımız var:



Burda yapmak istediklerimizden ilki yönetim vlan'ını 19 olarak ayarlamak. Normalde yönetim vlanı 1 'dir. Normalde interface vlan 1 diyip IP adresi veriyorduk. Bu şekilde switch'lere erişebiliyorduk. Burda yönetim vlan'ını 19'a taşıyacağız.

2. olarak switch'ler arası dinamik trunk protokolünü kapatıp manuel olarak trunk yapılandırmasını yapmak. 3. Olarak izin verilen vlan'ları tanımlayacağız, yani hangi bilgisayarın pingleyip pingleyemeyeceğini belirleyeceğiz. Mesela sadece vlan 19'daki bilgisayarlar switch'lere telnet yapabilecek. Sonrasında native vlan'ı değiştireceğiz, kullanılmayan port'ları kapatacağız, çünkü kötü niyetli biri fiziksel olarak switch'e bağlanırsa otomatik olarak vlan 1 'de olmuş olur. Kullanılmayan portları kapattıktan sonra blackhole denilen bir vlan atıyoruz. Bilinmeyen MAC adresleri bu blackhole vlan'a yönlendirilir, diğer vlan'lara erişememesi için.

Yapılandırmaya başlayalım. Henüz switch'lere sanal bir IP adresi atanmadı fakat bilgisayarların IP adresi atandı. Öncelikle belirlenen VLAN'ları iki switch'de de atayalım:

Switch0:

```
Switch0(config)#vlan 10
Switch0(config-vlan)#name IT
Switch0(config-vlan)#vlan 19
Switch0(config-vlan)#name Yonetim
Switch0(config-vlan)#vlan 20
Switch0(config-vlan)#name HR
Switch0(config-vlan)#vlan 99
Switch0(config-vlan)#name Native
Switch0(config-vlan)#vlan 899
Switch0(config-vlan)#name Karadelik
Switch0(config-vlan)#exit
```

```
10      IT      active
19      Yonetim active
20      HR      active
99      Native  active
899     Karadelik active
```

Switch1:

```
Switch1(config)#vlan 10
Switch1(config-vlan)#name IT
Switch1(config-vlan)#vlan 19
Switch1(config-vlan)#name Yonetim
Switch1(config-vlan)#vlan 20
Switch1(config-vlan)#name HR
Switch1(config-vlan)#vlan 99
Switch1(config-vlan)#name Native
Switch1(config-vlan)#vlan 899
Switch1(config-vlan)#name Karadelik
Switch1(config-vlan)#exit
```

```
10      IT      active
19      Yonetim active
20      HR      active
99      Native  active
899     Karadelik active
```

Şimdi vlan 10 ve 20 'yi ilgili portlara atayalım:

Switch0:

```
Switch0(config)#interface fastEthernet 0/1
Switch0(config-if)#switchport access vlan 10
Switch0(config-if)#exit
Switch0(config)#interface fastEthernet 0/2
Switch0(config-if)#switchport access vlan 20
Switch0(config-if)#exit
```

10	IT	active	Fa0/1
19	Yonetim	active	
20	HR	active	Fa0/2

Switch1:

```
Switch1(config)#interface fastEthernet 0/1
Switch1(config-if)#sw ac vl 10
Switch1(config-if)#exit
Switch1(config)#int fa0/2
Switch1(config-if)#sw ac vl 20
Switch1(config-if)#exit
```

10	IT	active	Fa0/1
19	Yonetim	active	
20	HR	active	Fa0/2

Şimdi de switch'lerin interface IP adreslerini verelim. Daha öncesinde interface vlan 1 şeklinde veriyorduk. Şimdi örnekte bahsettiğimiz gibi vlan 19 şeklinde vereceğiz.

Not: interface vlan 1 dediğimizde no sh komutu ile açmamız gerekiyordu, bu şekilde gerek yok, direk up durumuna geçer. Ama biz yine de aşağıda no sh dedik.

Telnet için parola da verildi.

```
Switch0(config)#interface vlan 19
Switch0(config-if)#
%LINK-5-CHANGED: Interface Vlan19, changed state to up

Switch0(config-if)#ip add 192.168.19.8 255.255.255.0
Switch0(config-if)#no sh
Switch0(config-if)#exit
Switch0(config)#line vty 0 4
Switch0(config-line)#pas cisco
Switch0(config-line)#login
Switch0(config-line)#exit
Switch0(config)#enable password cisco
Switch0(config)#end
```

Switch1 için de yapalım:

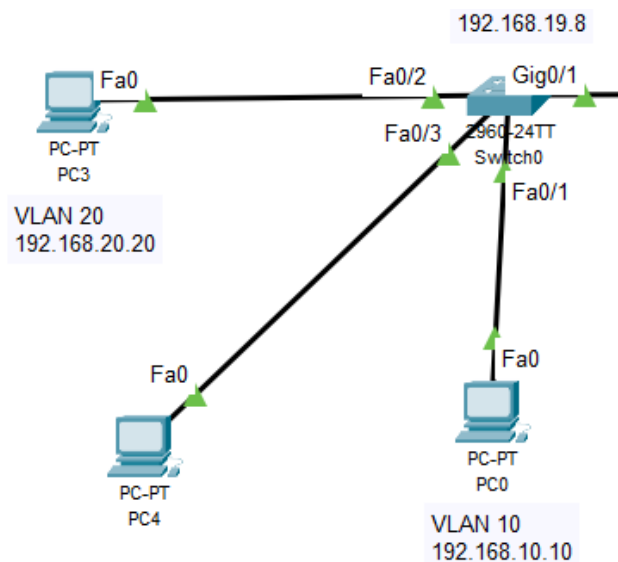
```

Switch1(config)#interface vlan 19
Switch1(config-if)#
%LINK-5-CHANGED: Interface Vlan19, changed state to up

Switch1(config-if)#ip add 192.168.19.9 255.255.255.0
Switch1(config-if)#exit
Switch1(config)#line vty 0 4
Switch1(config-line)#password cisco
Switch1(config-line)#exit
Switch1(config)#enable password cisco
Switch1(config)#end

```

Aşağıda vlan 20 deki ve vlan 10 'daki bilgisayarlar switch'e telnet yapamazlar, fakat yeni bir bilgisayar getirip switch'in fa0/3 portuna bağlarsak ve o portu da vlan 19'a atarsak bağlanabiliriz.



```

Switch0(config)#int fa0/3
Switch0(config-if)#sw ac vl 19
Switch0(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan19, changed state to up

```

Tabi bilgisayara 192.168.19.0 networkünden bir IP vermeyi de unutmayın. Görüldüğü gibi bağlanabiliyor:

```

C:\>telnet 192.168.19.8
Trying 192.168.19.8 ...Open

User Access Verification

Password:

```

Tabi şuan PC4 üzerinden Switch1'e ping atamıyoruz çünkü arada vlan'ları taşıyacak bir trunk yapılandırması yapmadık.

```
C:\>ping 192.168.19.9

Pinging 192.168.19.9 with 32 bytes of data:

Request timed out.
```

Switch'ler arası DTP kapatıp kendimiz manuel olarak TRUNK yapılandırmasını yapacağız:

Switch0:

```
Switch0(config-if)#interface gigabitEthernet 0/1
Switch0(config-if)#sw mo tr
Switch0(config-if)#switchport nonegotiate
Switch0(config-if)#
```

Switch1:

```
Switch1(config)#interface gigabitEthernet 0/1
Switch1(config-if)#sw mo tr
Switch1(config-if)#switchport nonegotiate
Switch1(config-if)#exit
```

Artık bu durumda PC4, Switch1'e de telnet yapabilecek:

```
C:\>telnet 192.168.19.9
Trying 192.168.19.9 ...Open

User Access Verification

Password: |
```

Şimdi native vlan'ı atayalım.

Native Vlan: bir trunk portunda etiketsiz (untagged) gelen/giden trafiğin ait olduğu vlan'dır. Trunk link'te **802.1Q** etiketi olmayan paketler native VLAN olarak işlenir. Karşı switch'te aynı native VLAN olmalı, yoksa **native VLAN mismatch** hatası olur.

Switch0: (eşleşmediğine dair hata verdi, o yüzden switch1'de de ayarlayalım.)

```
Switch0(config)#interface gigabitEthernet 0/1
Switch0(config-if)#switchport trunk native vlan 99
Switch0(config-if)%%SPANTREE-2-RECV_PVID_ERR: Received BPDU with inconsistent peer vlan id 1 on
GigabitEthernet0/1 VLAN99.

%SPANTREE-2-BLOCK_PVID_LOCAL: Blocking GigabitEthernet0/1 on VLAN0099. Inconsistent local vlan.
```

Switch1:

```
Switch1(config-if)#switchport trun
Switch1(config-if)#switchport trunk native vlan 99
Switch1(config-if)#
```

Şöyle bir şey yapalım. Trunk üzerinden sadece VLAN 10'un geçmesine izin verelim. VLAN 20'deki bilgisayarlar birbirini pingleyemesin, bunu sadece bir switch'de yapmamız yeterli olacaktır, sonuçta paket geçemeyecek:

```
Switch0(config)#interface gig
Switch0(config)#interface gigabitEthernet 0/1
Switch0(config-if)#switchport trunk allowed vlan 10
Switch0(config-if)#exit
```

Görüldüğü gibi geçmedi:

```
C:\>ping 192.168.20.21

Pinging 192.168.20.21 with 32 bytes of data:

Ping statistics for 192.168.20.21:
    Packets: Sent = 1, Received = 0, Lost = 1 (100% loss),

Control-C
```

Son olarak kullanmadığımız portları kapatıp bir karadelik vlan'ına atıyoruz. Mesela Switch0'da 20 ile 24 portlarının kullanılmadığını varsayalım.

```
Switch0(config)#interface range fas
Switch0(config)#interface range fastEthernet 0/20-24
Switch0(config-if-range)#sh

%LINK-5-CHANGED: Interface FastEthernet0/20, changed state to administratively down
%LINK-5-CHANGED: Interface FastEthernet0/21, changed state to administratively down
%LINK-5-CHANGED: Interface FastEthernet0/22, changed state to administratively down
%LINK-5-CHANGED: Interface FastEthernet0/23, changed state to administratively down
%LINK-5-CHANGED: Interface FastEthernet0/24, changed state to administratively down
Switch0(config-if-range)#switchport access vlan 899
```

Eğer biri bu portlara bağlansa bile (açık olduğunu varsay), karadelik vlan'ından olduğu için diğer vlan'lara erişemez.

899	Karadelik	active	Fa0/20, Fa0/21, Fa0/22, Fa0/23 Fa0/24
-----	-----------	--------	--

Sağdaki switch (switch1) 'de de aynı portları kapatalım ve karadelik vlan'ına atayalım:

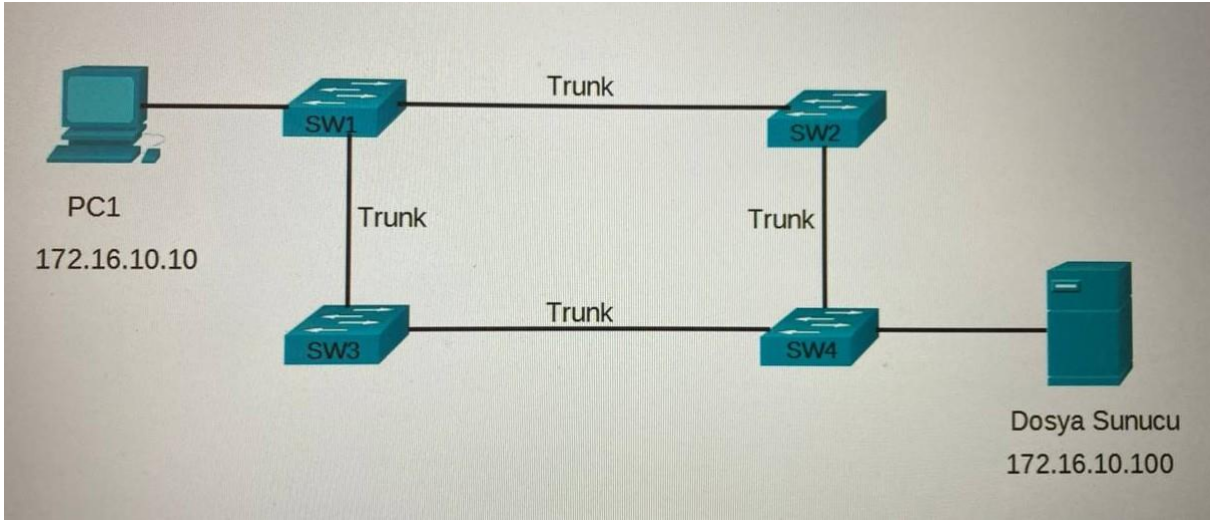
```
Switch1(config)#interface range fastEthernet 0/20-24
Switch1(config-if-range)#sh

%LINK-5-CHANGED: Interface FastEthernet0/20, changed state to administratively down
%LINK-5-CHANGED: Interface FastEthernet0/21, changed state to administratively down
%LINK-5-CHANGED: Interface FastEthernet0/22, changed state to administratively down
%LINK-5-CHANGED: Interface FastEthernet0/23, changed state to administratively down
%LINK-5-CHANGED: Interface FastEthernet0/24, changed state to administratively down
Switch1(config-if-range)#switchport access vlan 899
```

Bir LAN'da Fiziksel Artıklık

LAN'da fiziksel artıklık (**physical redundancy**), ağın fiziksel yapısında birden fazla bağlantı veya cihaz kullanılarak **yedekli (redundant)** bir yapı kurulması anlamına gelir. Yani amaç, bir kablo, anahtar (switch) ya da bağlantı **arızalansa bile** ağın çalışmaya devam etmesini sağlamaktır.

Daha basit tanımı, bir şey bozulduğunda bile ağın çalışmaya devam etmesini sağlamak için fazladan bağlantı veya cihaz bulundurmaktır.



Yukardaki yapıda örnek verecek olursak, eğer SW1 ile SW2 arasında bir bozukluk olursa, PC1, dosya sunucusuna SW1, SW3, SW4 üzerinden ulaşacaktır.

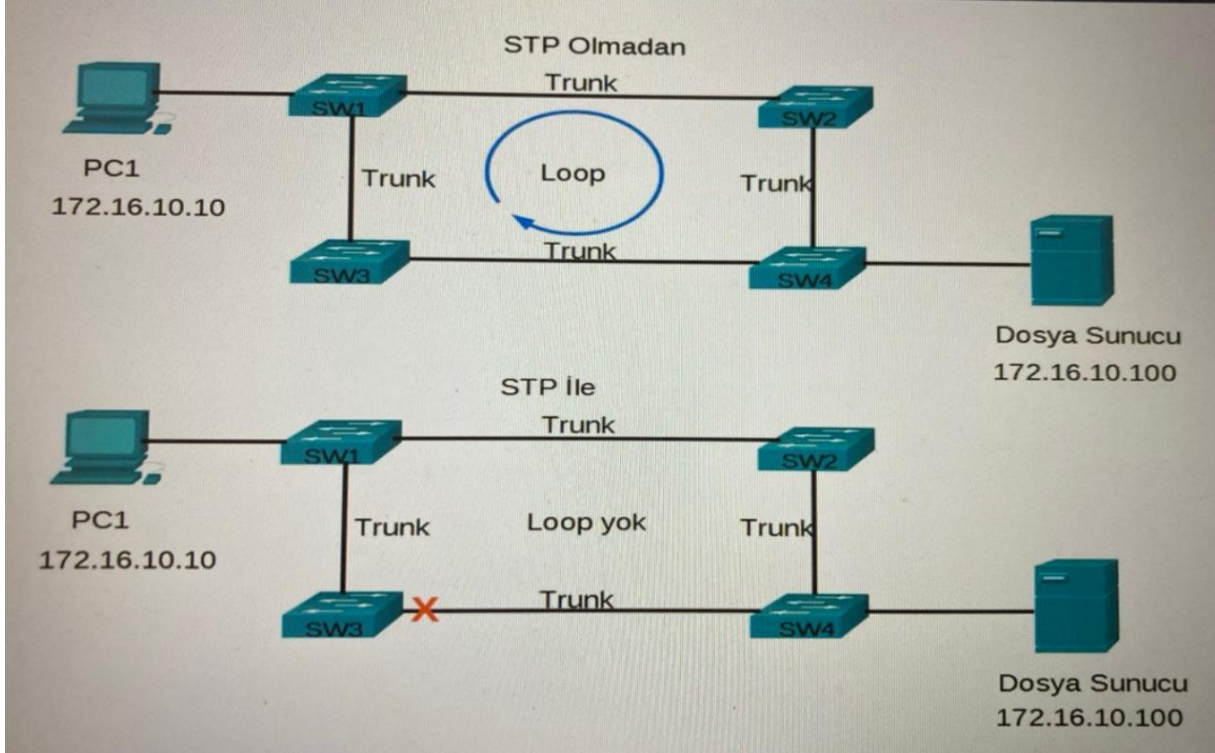
Yukardaki gibi fiziksel artıklık sağlamak için switch'ler arası birden fazla kabloyla bağladığımızı düşünelim. Bu yapıda Ethernet Frame için birden fazla yol vardır. Ethernet, yayın (broadcast) temelli bir teknolojidir. Yani bazı çerçeveler (örneğin ARP istekleri, broadcast paketleri) tüm ağdaki cihazlara gönderilir. Bir switch, bir broadcast paketi aldığı anda onu bütün portlarına gönderir. Eğer ağda bir döngü varsa, bu paket sonsuz döngüye girer.

Örnekte SW1'e PC1 üzerinden bir broadcast mesajı geldiğini varsayalım, bu paket yukardaki yapıya göre switch'ler arası sonsuz döngüde sıkışacaktır. Bunun sonucunda ağ trafiği tıkanır, CPU ve bant genişliği dolar, Cihazlar iletişim kuramaz hale gelir.

Burada devreye **Spanning Tree Protokolü (STP)** girer. Bu protokol Ethernet ağlarında loop oluşumunu engellemek için geliştirilmiş bir Layer 2 protokolüdür.

Spanning Tree Protocol (STP)

STP (Spanning Tree Protocol), Layer 2 (Veri Bağlantı Katmanı) seviyesinde çalışan bir protokoldür. Amacı; Ağda fiziksel olarak birden fazla yol olsa bile, mantıksal olarak döngüsüz (loop-free) bir yapı oluşturmaktır.



Yani STP, fazla bağlantıları analiz eder, en uygun yolu aktif (forwarding) tutar, geri kalanını bloklar. Bu sayede loop oluşmaz, ama yedek bağlantılar gerektiğinde devreye girebilir.

Yukardaki örnekte ise SW4 ve SW3 arasındaki portu kapatmış, PC1'den çıkan paketler SW1-SW2-SW4 üzerinden Dosya Sunucusuna ulaşıyor. Ama eğer olası arıza durumunda diyelim SW2 arızalandı ya da aradaki bağlantı kayboldu, bu sefer kapatılan port, bağlantı geri açılır ve trafiğin devamı sürdürülür.

Her switch, belirli aralıklarla **BPDU** adında özel mesajlar gönderir. Switch'ler bu BPDU'ları kullanarak ağın topolojisini keşfeder. Bu mesajda şunlar yer alır; Switch'in kimliği (Bridge ID), Root Bridge olduğunu iddia edip etmediği, Root'a olan yol maliyeti (path cost), Port bilgileri.

Spanning Tree (STP) İşlemleri

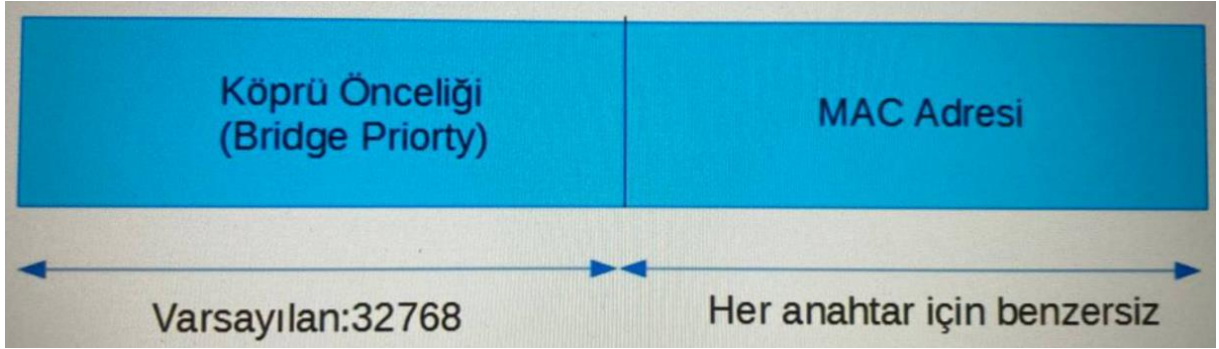
- Root bridge seçimi
- Root port seçimi
- Designated port seçimi
- Bloklanacak veya iletim durumunda olacak port seçimi

Root Bridge (Kök Anahtar) Seçimi: Ağda bir tane **Root Bridge** (kök switch) seçilir. Bu, ağın “merkezi referans noktası” gibidir. Seçim şu şekilde yapılır:

1. Her switch kendisini Root ilan eder ve BPDU gönderir.
2. Switch'ler BPDU'ları karşılaştırır.
3. **En düşük Bridge ID** değerine sahip olan Root olur.

Bridge ID = **Priority (öncelik) + MAC Adresi**

Eğer priority değerleri aynıysa, **MAC adresi küçük olan Root seçilir**. Default Priority değeri genellikle **32768**'dir.



Root Path Cost (Yol Maliyeti) Hesaplanır: Her switch, Root Bridge'e ulaşmak için en kısa (en düşük maliyetli) yolu bulur. Bu maliyet port hızına göre belirlenir.

Port Hızı	STP Path Cost
10 Mbps	100
100 Mbps	19
1 Gbps	4
10 Gbps	2

Switch, Root'a giden en düşük maliyetli yolu Root Port olarak işaretler.

Designated Port ve Blocking Port Belirlenir: Her ağ segmentinde:

- **Root Port:** Root Bridge'e en kısa yolu sağlayan port
- **Designated Port:** Segment üzerindeki en iyi (en düşük maliyetli) port
- **Blocking Port:** Döngüye neden olabilecek fazlalık bağlantılar – trafik iletilmez, sadece BPDU dinler.

Bu şekilde ağ **loop-free** hale gelir.

Eğer bir bağlantı koparsa, bir switch kapanırsa veya açılırsa, STP bu değişikliği fark eder. Daha önce **bloklanmış** bir yedek bağlantıyı devreye alır (forwarding'e geçirir). Yani ağ otomatik olarak kendini yeniden dengeler (reconverge eder).

Spanning Tree (STP) Çeşitleri

- **IEE 802.1 D** : İlk sürümdür. Amaç, Loop (döngü) önlemek. Yavaş toparlanır (30-50 saniye). Tek bir ağaç oluşturur.
- **CST**: Tüm VLAN'lar için tek bir STP instance'ı kullanır. Basittir ama yük paylaşımı yapılamaz.
- **PVST+**: Per VLAN STP, Cisco'ya özgüdür. Her VLAN için ayrı STP çalışır, yük paylaşımı yapılabilir. PVST+ sürümü 802.1Q VLAN etiketleme ile uyumludur.
- **802.1 S (MSTP)**: Multiple STP, birden fazla VLAN'ı tek STP grubu altında toplar. Hem hızlı (RSTP temelli) hem de ölçeklenebilir. Günümüz büyük ağlarında tercih edilir.
- **802.1 w (RSTP)**: STP'nin hızlı versiyonudur. Aynı mantıkla çalışır ama 1-2 saniyede toparlanır. Günümüzde en çok kullanılan sürümlerden biridir.
- **Rapid PVST +**: Cisco'ya özgü, RSTP temelli. Çok hızlı toplanır (1-2 saniye). Her vlan için ayrı STP oluşturur. Load balancing mümkün

Protokol	Standart	Kaynak İhtiyacı	Convergence*	STP Ağaç Sayısı
STP	802.1 D	Düşük	Yavaş	Bir tane
PVST+	Cisco	Yüksek	Yavaş	Her VLAN için bir tane
RSTP	802.1 w	Orta	Hızlı	Bir tane
Rapid PVST+	Cisco	Çok yüksek	Hızlı	Her VLAN için bir tane
MSTP	802.1 s, Cisco	Orta veya yüksek	Hızlı	Her çoklu VLAN için bir tane

Convergence: STP'nin çalışıp bütün seçimlerin bittiği, portların iletim ya da bloklandığı durumdur. Convergence (yakınsama) bitinceye kadar veri iletişimi durur.

STP – PVST+ (Kitap)

STP

Layer 2 Switching

Layer 2 switching teknolojisi donanıma dayalıdır. Bu nedenle host üzerindeki MAC adresini kullanarak network'ü filtreler. Switch'lerde tablolarını oluşturmak için ASIC (Application Specific Integrated Circuits) yöntemini kullanırlar. Cisco switch'lerde default olarak enable durumundadır.

Layer 2 switch aygıtları paketleri (frame) donanım adreslerine göre gönderdikleri için hızlıdır. Layer 2 özelliklerini şu şekilde özetleyebiliriz:

Donanım temelli köprü (bridging) / Hız / Düşük Maliyet

Layer 2 Switch Fonksiyonları

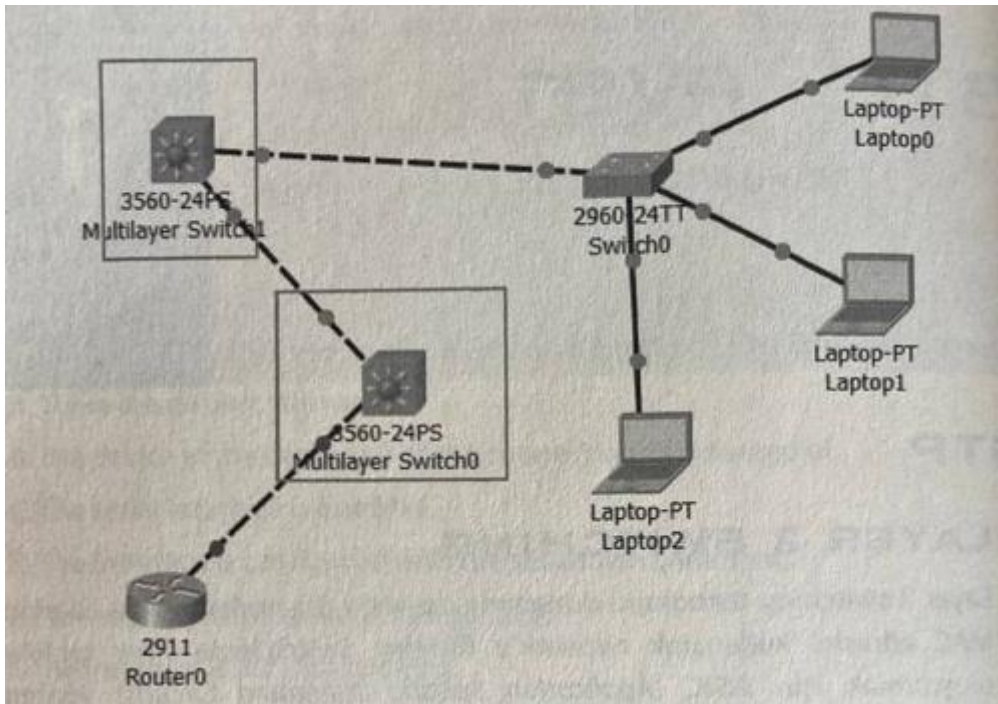
Layer 2 switching sürecinde üç işlem yerine getirilir:

Address Learning: Adres Öğrenme

Forwarding / Filtering: Yönlendirme / Filtreleme

Loop Avoidance: Döngü önleme

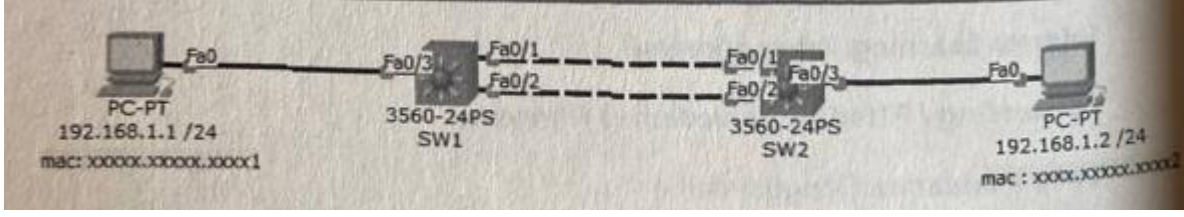
Adres öğrenme işlemi her paketin MAC adresinin, bir MAC veri tabanına tutulmasıyla sağlanır. Yönlendirme işlemi ise alınan paketin hedef adresinin bulunmasını sağlar. Döngü önleme özelliği ise, switch'ler arasında birden çok bağlantının olduğu durumlarda fazlalık (**redundancy**) oluşursa, bu özellik döngüyü önler. Döngü önlemede **STP** (Spanning Tree Protocol) kullanılır.



Yukarıdaki network'te işaretli switch'lerin bozulması durumunda tüm trafik aksayacaktır. Bunu önleme adına bilgisayarların bağlı olduğu switch'i diğer switch'lere doğrudan bağladığımızda ise loop olacaktır. İşte bunu önleme adına Spanning Tree Protocol varsayılan olarak tüm Cisco switch'lerde enable durumdadır.

Spanning tree protokolünün durumunu görüntülemek için;

```
Switch#sh spanning-tree
```



Üsteki bir şekilde bir yapıda soldaki pc sağdakine ulaşmak için aynı network'te olduğundan **Arp request** yapar. SW2 bu paketi aldığı anda hedefin broadcast olduğunu anlar (MAC ff doludur). Aldığı paketi diğer portundan tekrar SW1'e gönderir ve döngü oluşur.

Döngüleri önlemek için bir switch'in herhangi bir portu UP olduğunda switch hemen **BPDU** (Bridge Protocol Data Unit) gönderir. Tüm switchler her 2 saniyede bir bu paketleri gönderir. BPDU paketleri Bridge ID; Bridge Priority ve Mac Adress'inden oluşur ve 8 byte'dir. Switch bir porttan gönderdiği BPDU paketini başka bir portundan tekrar alırsa, anlar ki loop olmak üzere ve bu yeni UP olan portu aktifleştirmesin, (non-designated port) korumaya alır. Bu sefer hangi switch, hangi portu kapatacak konusu başlar.

Özetle STP (Spanning Tree Protocol), DEC tarafından geliştirilmiş ve IEEE tarafından 802.1 olarak kabul edilmiş bir protokoldür. STP'nin ana görevi Layer 2 network'lerinden oluşan döngüleri durdurmaktır. STP sürekli olarak network'ü izler ve gereksiz hatları kapatarak döngü oluşmamasını sağlar.

STP, network üzerindeki bütün bağlantıları bulur ve fazla (gereksiz) bağlantıları kapatır. Bu işlem network üzerinde bir **Root Bridge** seçilmesiyle olur. Bir network'ta yalnız bir root bridge vardır. Diğer switch'ler ise **Nonroot Bridge** olarak adlandırılır.

Switch'ler hiçbir yönlendirme protokolü kullanmazlar, Her switch, BPDU (Bridge Protocol Data Unit) mesajı yayın yaparak kendisinin root olduğunu söyler, önceliği düşük olan root seçilir, eşitlik durumunda ise düşük ID'li (veya düşük MAC adrese sahip olan) olan root seçilir. Root olan switch'in tüm portları her zaman açıktır ve yedek bağlantıları ve asıl bağlantıları belirler.

Bir switch'i manuel olarak root bridge yapmak için:

```
Switch(config)# spanning-tree priority 1
```

Bridge Priority default olarak 32768 değerindedir ve bizim tarafımızdan manuel olarak değiştirilebilir. Root, ID'leri aynıysa Root Bridge seçimi için Mac Address bilgisine bakacaklardır. Mac adresi en küçük olan switch Root Bridge olacaktır.

Spanning Tree çalışma prensibi içerisinde sırayla 4 adımdan bahsedebiliriz bunlar;

1. Root Bridge seçimi yapılır.
2. Switch'lerin Root Portları belirlenir.
3. Switch'lerin Designated Portları belirlenir.
4. Diğer Portlar Block (non designated port) durumda olacaktırlar.

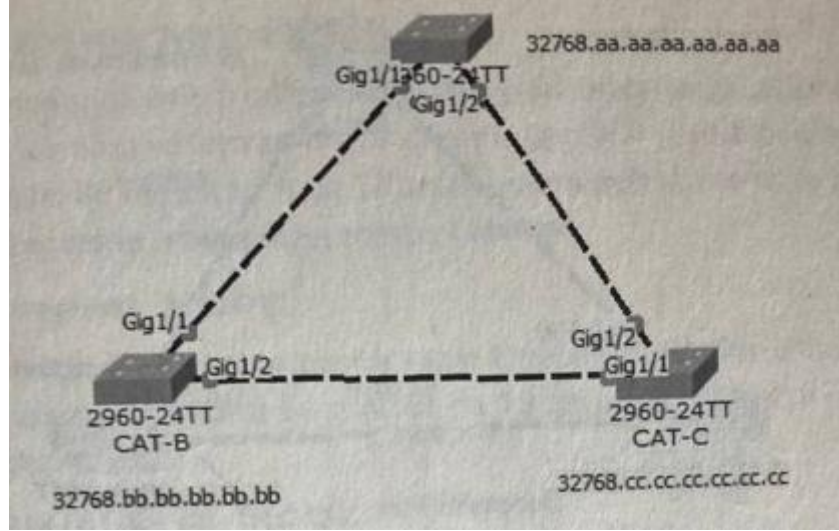
Root Bridge seçimini tamamladık, şimdi Switch'lerde Root Portlar nasıl seçilir? Bundan bahsedelim.

Root Portlar her switch için 1 tane olabilir ve Root Bridge'ye olan yakınlığına göre bir port, **Root Port** olur. Burada yakınlıktan kastedilen cost değeridir. Cost değeri 2 switch arasındaki hattın hızıyla orantılı olup sabit değerlerle ifade edilir.

IEEE'nin çok kullanılan bant genişlikleri için yayınladığı cost değerleri aşağıdaki tablodaki gibidir:

COST DEĞERLERİ	
4 Mbps	250
10 Mbps	100
16 Mbps	62
45 Mbps	39
100 Mbps	19
155 Mbps	14
1 Gbps	4
10 Gbps	2

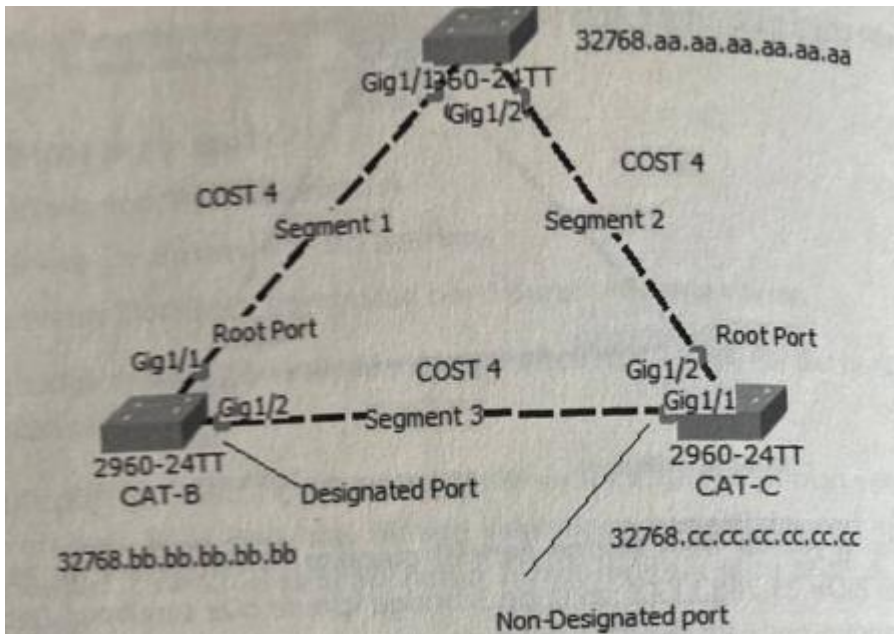
Root bridge'te cost'u en düşük olan port, root port olacaktır. Switch root bridge'ye 2 yoldan erişiyor ve cost'ları eşit olduğu durumda ise; switch'in port ID'sine bakılır. Port numarası küçük olan port (Örnek; FastEthernet 0/1 port numarası FastEthernet 0/2 den küçüktür.) Root Port olacaktır. Bir bridge üzerinde yalnızca bir tane Root Port olabilir.



Şekildeki yapıda Root Bridge hangisi olacaktır? 3 Bridge'miz var ve Bridge ID'leri BID=31.768.XXXX şeklinde 3 bridge için de bize sunulmuş. Daha önce belirttiğimiz gibi Bridge ID'si en küçük olan Root Bridge olacaktır. Yani Cat-A en üstteki switch, Root Bridge'dir. Çünkü Priority değerleri aynı olduğundan, MAC adresi küçük olan switch seçilir.

Şimdiye kadar Root Bridge seçimini ve Root Port seçimlerini öğrendik. Şimdi önemli bir konu olan ağda loop engellemek için gerektiğinde hangi portun kapatılacağını belirlenmesine (designated port) değinelim. Designated port seçimine de geçmeden önce bir kavramı açıklamak gerekir. 2 switch arasındaki yola **segment** adını veriyoruz. Designated port seçimi yaparken, her segment için yalnızca bir tane designated port olacağını unutmamalıyız. Ayrıca Root Bridge'nin tüm portları Designated Port durumundadır ve forwarding state olmalıdır. Designated port bir switch için birden fazla olabilir. (Root Port yalnızca 1 tane olabilir)

Bu bilgiler ışığında artık Designated portlarımızı belirleyebiliriz.



Şekilde en üstteki switch Root Bridge olacağından ve daha önce belirttiğimiz gibi tüm portlarının Designated olacağından Segment 1 ve Segment 2 için Designated Port'larımızı belirlemiş olduk. Bir segmentte yalnızca 1 tane Designated Port olacağını biliyoruz.

Segment 1 ve Segment 2 'ye baktığımızda Cat-B ve Cat-C için Switch'lerinin 1/1 interface'leri Root Port olmalıdır. Segment 3 için 1 tane Designated Port olmalıdır. Burada Root Port olamaz çünkü her iki switch içinde Root Port'lar belirlendi. Bir switch üzerinde yalnızca 1 tane Root Port olacağı bilgisini hatırlayalım. Cat-B MAC adresi, Cat-C switch'ine göre daha küçük olduğu için; Cat-B switch'inin Gi1/2 portu Segment 3 için Designated Port olacak ve Cat-C switch'inin Gi1/1 portu da Non-Designated Port durumunda olacaktır.

Spanning-Tree çalışan switch'ler de sırayla 4 farklı aşama uygulanır:

1-Blocking Mode

Switch bu mod'dayken port kapalı olarak düşünebiliriz. (Switch'de loop engellenir) Ancak unutmamalıyız ki bu mod'da switch her 2 sn aralıklarla **hello** paketlerini alacaktır ve eğer 20 sn hello paketi alamazsa bir sonraki mod olan **listening** moduna geçecektir.

2-Listening Mode

Listening modunda switch bu moddan sonraki adımlara geçip geçmeyeceğine karar verir, learning ve forwarding mod'larına geçebilir veya **Blocking** duruma geri dönebilir. Bu mod içerisinde **15** sn bulunur ve devam ederse bir sonraki mod olan **Learning** moduna geçecektir.

3-Learning Mode

Learning modunda switch dinleme yapar diyebiliriz (iletişim olmaz) ve MAC address table'sini oluşturur ve bu mod'da da 15 sn kaldıktan sonra Forwarding moduna geçecektir.

4-Forwarding Mode

Forwarding mod'unda switch artık diğer switch'lerle haberleşme halindedir (Frame alımı ve gönderimi yapılır). Forwarding modu için söyleyeceğimiz kritik bir not Root Bridge'nin bütün portları her zaman forwarding durumda olacaktır, yani Root Bridge'nin hiçbir portu STP tarafından kapatılamaz.

Toplamda Spanning Tree Blocking mod'dan Forwarding mod'a kadar 50 sn süre içerisinde çalışacaktır. Switch – Switch bağlantıları için bu süre kaçınılmazdır. Ancak switch'in diğer portlarına bağlı bir host için hostun bu süreyi beklemesi zorunlu değildir. Bunun için switch'lerde **portfast** komutu çalıştırılır. Bu komutu global konfigürasyon mod'unda kullanabileceğimiz gibi interface altında da kullanabiliriz. Global konfigürasyon modunda kullanılması önerilmez.

```
Switch(config)#interface fastethernet 0/1  
Switch(config-if)#spanning-tree portfast
```

PVST (Per Vlan Spanning Tree)

Switch'lerde, vlan'lar kullanılması durumunda Spanning Tree nasıl çalışacak bizim bunun için neler yapmamız gerektiğine bir göz atalım.

Network'ümüz de vlan'lar olabilir ve biz her vlan için ayrı bir Root Bridge atayabiliriz. Bu durum switch'lerde PVST (Per Vlan Spanning-Tree) olarak ifade edilir ve global konfigürasyon moduna:

```
Switch(config)#spanning-tree mode pvst
```

Komutuyla enable edilir.

```
Switch(config)#spanning-tree vlanid root priority
```

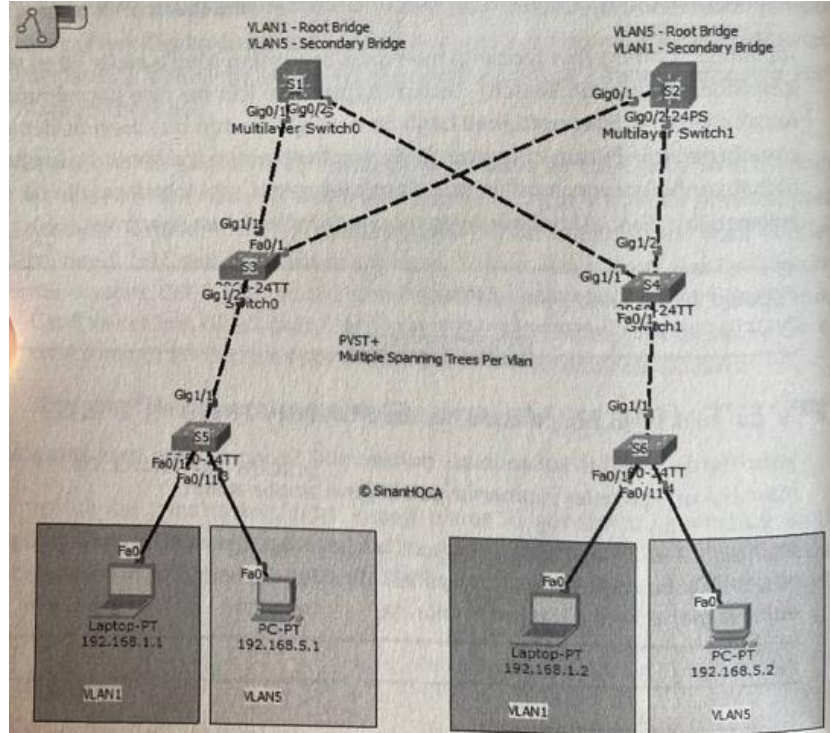
Komutu uyguladığımızda switch bu vlan ID için root bridge olacaktır ve yine yukarıda yazdığımız her şey bu switch portları ve bu vlan için geçerli olacaktır. Vlan bazında STP uygulamak için PVST uygulamak bize bir switch üzerindeki bir portun devamlı olarak block durumda olmasını engellemek açısından faydalı olacaktır.

Ayrıca;

```
Switch(config)#spanning-tree vlanid root secondary
```

Komutuyla switch belirtilen vlan, ID için backup bridge olarak çalıştırılabilir.

Şimdi PVST konusunu bir örnek ile açıklayalım:



Yukarıdan başlayarak ayarları yapalım: Öncelik S1 switch'ine sen vlan1'in paşasısın (Root Bridge) diyelim ve gerekli ayarları yapalım sonra da kontrolümüzü yapalım:

```
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#hostname S1
S1(config)#int gi 0/1
S1(config-if)#switchport trunk encapsulation dot1q
S1(config-if)#switchport mode trunk
```

```
S1(config)#int gi 0/2
S1(config-if)#switchport trunk encapsulation dot1q
S1(config-if)#switchport mode trunk
```

```
S1(config)#vlan 5
S1(config-vlan)#ex
S1(config)#do sh vl
```

VLAN	Name	Status	Ports
1	default	active	Fa0/1, Fa0/2, Fa0/3, Fa0/4 Fa0/5, Fa0/6, Fa0/7, Fa0/8 Fa0/9, Fa0/10, Fa0/11, Fa0/12 Fa0/13, Fa0/14, Fa0/15, Fa0/16 Fa0/17, Fa0/18, Fa0/19, Fa0/20 Fa0/21, Fa0/22, Fa0/23, Fa0/24
5	VLAN0005	active	
1002	fddi-default	act/unsup	
1003	token-ring-default	act/unsup	
1004	fddinet-default	act/unsup	
1005	trnet-default	act/unsup	

```
S1(config)#spanning-tree mode pvst
S1(config)#spanning-tree vlan 1 root primary
S1(config)#spanning-tree vlan 5 root secondary
```

```
S1#sh spanning-tree
VLAN0001
```

Spanning tree enabled protocol ieee

Root ID Priority 24577

Address 000C.CF01.2579

This bridge is the root

Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec

Bridge ID Priority 24577 (priority 24576 sys-id-ext 1)

Address 000C.CF01.2579

Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec

Aging Time 20

Interface	Role	Sts	Cost	Prio.Nbr	Type
-----------	------	-----	------	----------	------

Gi0/2	Desg	FWD	4	128.26	P2p
-------	------	-----	---	--------	-----

Gi0/1	Desg	FWD	4	128.25	P2p
-------	------	-----	---	--------	-----

VLAN0005

Spanning tree enabled protocol ieee

Root ID Priority 28677

Address 000C.CF01.2579

This bridge is the root

Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec

Bridge ID Priority 28677 (priority 28672 sys-id-ext 5)

Address 000C.CF01.2579

Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec

Aging Time 20

Interface	Role	Sts	Cost	Prio.Nbr	Type
-----------	------	-----	------	----------	------

Gi0/2	Desg	FWD	4	128.26	P2p
-------	------	-----	---	--------	-----

Gi0/1	Desg	FWD	4	128.25	P2p
-------	------	-----	---	--------	-----

Bu ayarlardan sonra baktık ki her iki vlan da paşalığını ilan etmiş. Diğerlerinin ayarlarını yapmadığımız için böyle oldu.

Şimdi ikinci switch ayarlarını da yapalım bakalım diğer vlan için de paşalığını sürdürebilecek mi?

```
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#hostname S2
S2(config)#int gi 0/1
S2(config-if)#sw
S2(config-if)#switchport tr
S2(config-if)#switchport trunk en
S2(config-if)#switchport trunk encapsulation d
S2(config-if)#switchport trunk encapsulation dot1q
S2(config-if)#sw
S2(config-if)#switchport mod
S2(config-if)#switchport mode tr
S2(config-if)#switchport mode trunk

S2(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1,
changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1,
changed state to up

S2(config-if)#ex
S2(config)#int gi 0/2
S2(config-if)#sw
S2(config-if)#switchport mod
S2(config-if)#switchport tr
S2(config-if)#switchport trunk en
S2(config-if)#switchport trunk encapsulation d
S2(config-if)#switchport trunk encapsulation dot1q
S2(config-if)#sw
S2(config-if)#switchport mo
S2(config-if)#switchport mode tr
S2(config-if)#switchport mode trunk

S2(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2,
changed state to down
```

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2,
changed state to up

ex

S2(config)#sp

S2(config)#spanning-tree mo

S2(config)#spanning-tree mode pv

S2(config)#spanning-tree mode pvst

S2(config)#sp

S2(config)#spanning-tree ?

mode Spanning tree operating mode

portfast Spanning tree portfast options

vlan VLAN Switch Spanning Tree

S2(config)#spanning-tree vlan 5 ?

priority Set the bridge priority for the spanning tree

root Configure switch as root

<cr>

S2(config)#spanning-tree vlan 5 root

S2(config)#spanning-tree vlan 5 root ?

primary Configure this switch as primary root for this spanning
tree

secondary Configure switch as secondary root

S2(config)#spanning-tree vlan 5 root pr

S2(config)#spanning-tree vlan 5 root primary

S2(config)#sp

S2(config)#spanning-tree vlan 1 ro

S2(config)#spanning-tree vlan 1 root sec

S2(config)#spanning-tree vlan 1 root secondary

S2(config)#ex

S2#

%SYS-5-CONFIG_I: Configured from console by console

S2#sh sp

S2#sh spanning-tree

VLAN0001

Spanning tree enabled protocol ieee

Root ID	Priority	24577
---------	----------	-------

Address	000C.CF01.2579
---------	----------------

Cost	8
------	---

Port	26(GigabitEthernet0/2)
------	------------------------

```

Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
Bridge ID Priority 28673 (priority 28672 sys-id-ext 1)
Address 0001.42A7.4C3C
Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
Aging Time 20

```

Interface	Role	Sts	Cost	Prio.Nbr	Type
Gi0/1	Altn	BLK	19	128.25	P2p
Gi0/2	Root	FWD	4	128.26	P2p

```

S2#conf t
Enter configuration commands, one per line. End with CNTL/Z.

```

```

S2(config)#vlan 5
S2(config-vlan)#ex
S2(config)#end

```

```

S2#
%SYS-5-CONFIG_I: Configured from console by console

```

```
S2#
```

```
S2#
```

```
S2#sh sp
```

```
S2#sh spanning-tree
```

```
VLAN0001
```

```
Spanning tree enabled protocol ieee
```

```
Root ID Priority 24577
```

```
Address 000C.CF01.2579
```

```
Cost 8
```

```
Port 26(GigabitEthernet0/2)
```

```
Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
```

```
Bridge ID Priority 28673 (priority 28672 sys-id-ext 1)
```

```
Address 0001.42A7.4C3C
```

```
Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
```

```
Aging Time 20
```

Interface	Role	Sts	Cost	Prio.Nbr	Type
Gi0/1	Altn	BLK	19	128.25	P2p
Gi0/2	Root	FWD	4	128.26	P2p

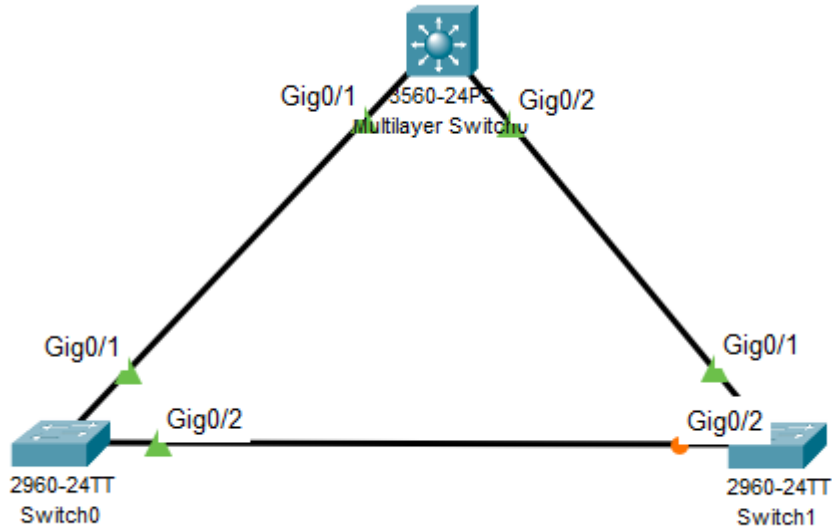
```
VLAN0005
Spanning tree enabled protocol ieee
Root ID    Priority    24581
           Address    0001.42A7.4C3C
           This bridge is the root
           Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec

Bridge ID   Priority    24581 (priority 24576 sys-id-ext 5)
           Address    0001.42A7.4C3C
           Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
           Aging Time 20

Interface    Role Sts Cost      Prio.Nbr Type
-----
Gi0/1        Desg LSN 19      128.25 P2p
Gi0/2        Desg LSN 4       128.26 P2p
```

STP Yapılandırması

Aslında Cisco switch'lerde Spanning Tree etkin olarak gelir, herhangi bir yapılandırma yapmamıza gerek kalmaz.



Yukardaki gibi bir yapıda STP çalıştı ve portları falan belirledi. Görüldüğü gibi Switch1'in Gig0/2 portunu kapattı ki loop oluşmasın. Eğer orayı kapatmasaydı da yeşil yansaydı, loop'a sebep olurdu. Peki acaba Root Bridge olarak hangisini seçti? Switch 1 'in bir portu kapalı olduğuna göre o değil. Kalan 2 switch'ten biri olmalı. Bizim istediğimiz Distribution Layer'daki L3 Switch'in Root Bridge olarak seçilmesidir. Eğer değilse biz kendimiz değiştiririz.

Önceki soldaki (Switch0) switch'e bakalım. Show spanning-tree komutu ile STP detaylarına bakalım:

```
Switch#show spanning-tree
VLAN0001
  Spanning tree enabled protocol ieee
  Root ID    Priority    32769
             Address     0001.C98C.2A57
             Cost        4
             Port        25 (GigabitEthernet0/1)
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID  Priority    32769 (priority 32768 sys-id-ext 1)
             Address     000A.41AE.6E27
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
             Aging Time  20

Interface                Role Sts Cost      Prio.Nbr Type
-----
Gi0/1                    Root FWD 4         128.25  P2p
Gi0/2                    Desg FWD 4         128.26  P2p

Switch#
```

Yukardaki Root ID'deki Root Bridge seçilen Switch bilgileri, altındaki Bridge ID ise baktığımız Switch'in bilgileri. İkisinin MAC adresleri farklı olduğunu görüyoruz. Demek ki Root Bridge olarak L3 Switch'i seçmiş. Gerçekten de öyle mi bakalım:

Evet, hem MAC adreslerin aynı olduğunu hem de "This bridge is the root" şeklinde belirttiğini görürüz.

```
Switch#show spanning-tree
VLAN0001
  Spanning tree enabled protocol ieee
  Root ID    Priority    32769
             Address     0001.C98C.2A57
             This bridge is the root
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID  Priority    32769 (priority 32768 sys-id-ext 1)
             Address     0001.C98C.2A57
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
             Aging Time  20

Interface                Role Sts Cost      Prio.Nbr Type
-----
Gi0/1                    Desg FWD 4         128.25  P2p
Gi0/2                    Desg FWD 4         128.26  P2p

Switch#
```

Demek ki en düşük MAC adresi bundaymış ki Root Bridge olarak seçilmiş. Soldaki Switch'in Root Bridge'e giden portun root olduğunu görürüz. Ayrıca Cost değerlerinin Gigabit port olduğundan ötürü 4 değerinde olduğunu görürüz. FastEthernet olsaydı cost değeri daha fazla olurdu.

Yukarda zaten Root Bridge otomatik olarak istediğimiz gibi seçilmiş. Fakat olmasaydı biz nasıl değiştirdik? Bunun için 2 yöntemimiz var, ilki şu şekilde;

Tabi daha vlan yapılandırmamız yok, olsaydı seçebilirdik. Sonrasında ya priority değeri veriyoruz ya da root primary veya root secondary olarak seçebiliyoruz.

Eğer root primary dersek varsayılan priority değerini 24576'ya çeker. Bunu duruma göre BPDU mesajlarına göre daha düşük bir değere ayarlayabilir. Root Secondary komutunu kullandığımızda ise priority değerini 28672'ye ayarlar. Fakat gerçek uygulamalarda bu şekilde değil de diğer türlü yapılması tercih edilebilir. 0 ile 61440 arasında priority değeri ayarlanır, 4096'nın katları şeklinde belirlenir.

```
Switch(config)#spanning-tree vlan ?
WORD vlan range, example: 1,3-5,7,9-11
Switch(config)#spanning-tree vlan 1 ?
priority Set the bridge priority for the spanning tree
root Configure switch as root
<cr>
Switch(config)#spanning-tree vlan 1 root ?
primary Configure this switch as primary root for this spanning tree
secondary Configure switch as secondary root
Switch(config)#spanning-tree vlan 1 priority ?
<0-61440> bridge priority in increments of 4096
Switch(config)#spanning-tree vlan 1 priority 0
Switch(config)#
```

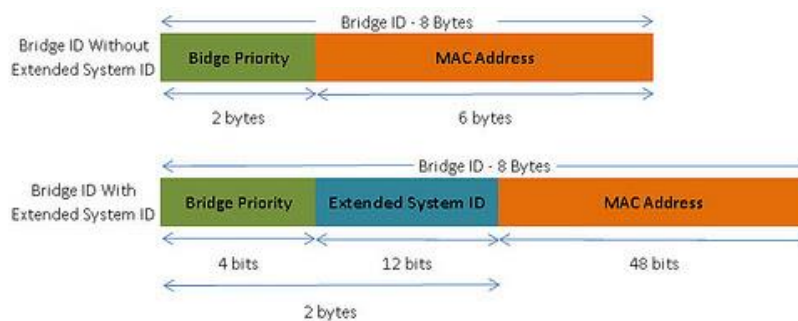
Örnek olması açısından 0 verip geçebiliriz. Fakat priority değeri 1 gösterecektir. Çünkü vlan id bilgisi de eklendi.

```
Switch#show spanning-tree
VLAN0001
Spanning tree enabled protocol ieee
Root ID    Priority    1
Address    0001.C98C.2A57
This bridge is the root
Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec

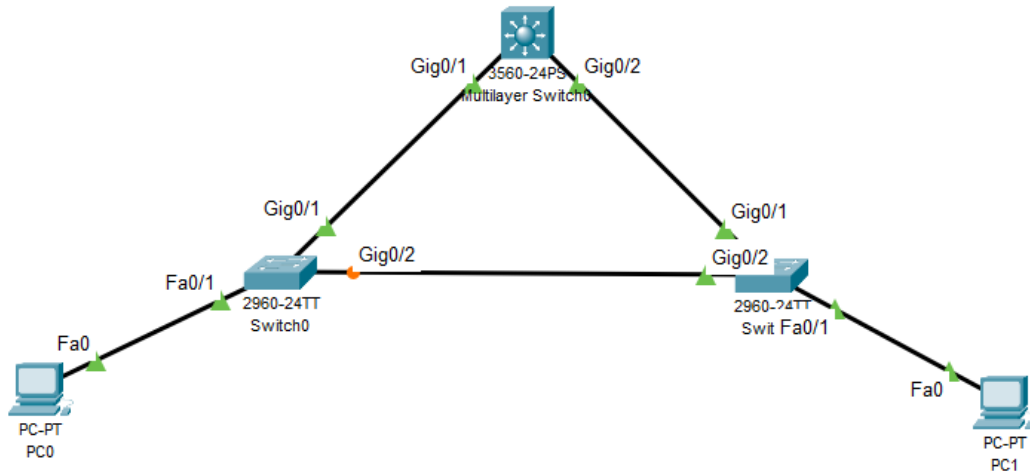
Bridge ID   Priority    1 (priority 0 sys-id-ext 1)
Address     0001.C98C.2A57
Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
Aging Time 20

Interface   Role Sts Cost      Prio.Nbr Type
-----
Gi0/1       Desg FWD 4         128.25 P2p
Gi0/2       Desg FWD 4         128.26 P2p

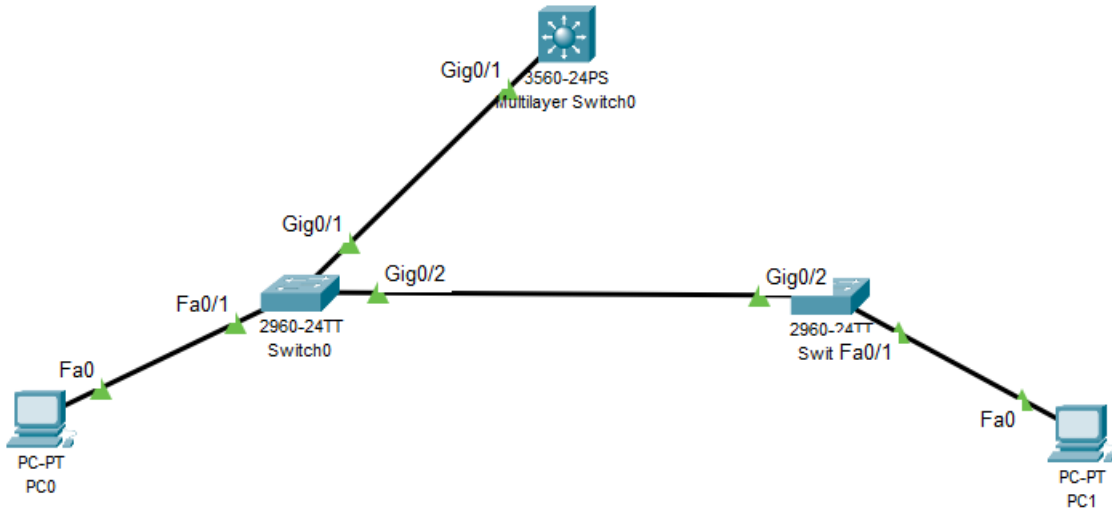
Switch#
```



Rapid STP Yapılandırması



Yukardaki gibi bir yapıda PC0, PC1'e ulaşmak için sırasıyla Switch0, MultiLayer Switch ve Switch1 üzerinden gidecektir. Fakat Multilayer Switch ile Switch1 arasındaki bağlantı bozulursa Alternatif yol olacak olan Switch0 üzerindeki Gig0/2 portu Spanning Tree durumlarından dolayı 50 saniye sonra aktif olur.

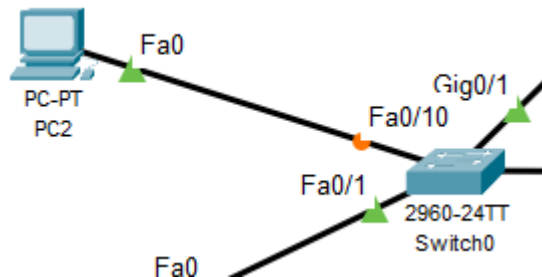


Eğer bu süre bize yavaş gelirse, STP'nin Rapid modunu ayarlayabiliriz. Tabi bunu tüm switch'lerde yapmamız lazım. Bu bekleme süresini yaklaşık 10-15 saniyeye kadar düşürecektir.

```
Switch(config)#sp
Switch(config)#spanning-tree mode ?
  pvst          Per-Vlan spanning tree mode
  rapid-pvst    Per-Vlan rapid spanning tree mode
Switch(config)#spanning-tree mode rapid-pvst
Switch(config)#
```

STP Portfast ve BPDU Guard Yapılandırması

Bir önceki yapıda Switch0'a bir bilgisayar bağlarsak STP durumundan dolayı bir süre sonra bağlantı sağlanacaktır. Bunun yaşanmasını istemiyorsak ve anında bağlantı sağlamasını istiyorsak, bilgisayarın bağlı olduğu portu **portfast** olarak ayarlayabiliriz.



Bunun için ilgili portun arayüzüne gireriz;

```
Switch(config)#interface fastEthernet 0/10
Switch(config-if)#spanning-tree portfast ?
  disable  Disable portfast for this interface
  trunk    Enable portfast on the interface even in trunk mode
  <cr>
Switch(config-if)#spanning-tree portfast
%Warning: portfast should only be enabled on ports connected to a single
host. Connecting hubs, concentrators, switches, bridges, etc... to this
interface when portfast is enabled, can cause temporary bridging loops.
Use with CAUTION

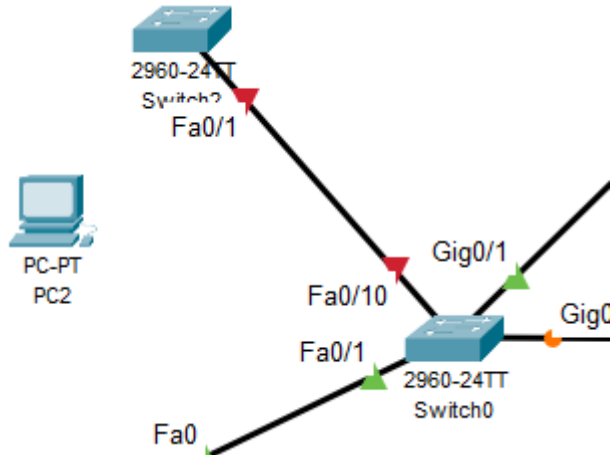
%Portfast has been configured on FastEthernet0/10 but will only
have effect when the interface is in a non-trunking mode.
Switch(config-if)#
```

Fakat bunu yaptığımızda bizi uyarır; portfast olarak ayarladığımız porta host dışında (PC, IP telefon, Yazıcı) bir cihaz bağlanmamasını (Hub, Switch..) söyler. Çünkü bu durumda Loop'a girebilir.

Bunun için portfast'in yanında bir yapılandırma daha yapılır, buna da **BPDU Bodyguard** denir. Eğer oldu da portfast ayarlanan porta bir Switch vs takılırsa, ve bu switch'ten bir BPDU mesajı gelirse, anında o portu kapatacak ve error disable moduna düşecek. Bunu aktif etmek için;

```
Switch(config-if)#sp
Switch(config-if)#spanning-tree bpduguard enable
Switch(config-if)#
```

Bunu denemek adına fa0/10 portuna bir switch takalım. Portun kapandığını görürüz;

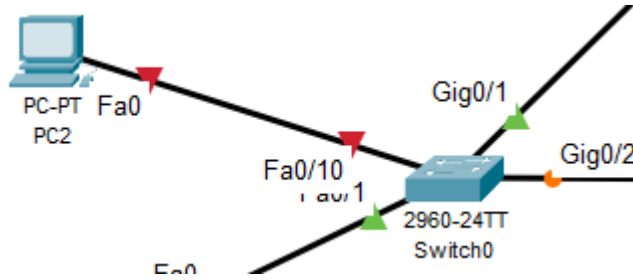


Switch'teki bu hatadan da bunu görürüz;

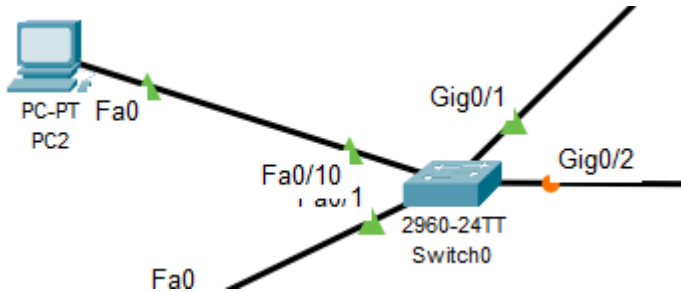
```
state to up
%SPANTREE-2-BLOCK_BPDUGUARD: Received BPDU on port FastEthernet0/10 with BPDU
Guard enabled. Disabling port.

%PM-4-ERR_DISABLE: bpduguard error detected on 0/10, putting 0/10 in err-
disable state
```

Olurda kabloyu çıkarıp tekrar bilgisayara takarsak port aktif olur mu? Hayır. Bizim o portu tekrar aktif etmemiz lazım.



Bunun için önce ilgili port'un arayüzüne girip, önce shutdown edip sonra tekrar no shutdown ile portu kurtarıyoruz. Bu sayede tekrar bağlantı sağlanır.



Otomatik kurtarma komutları da var fakat ccna konusu değil, zaten packet tracer'da desteklemez.

Etherchannel

Etherchannel (Bağlantı Kümeleme) iki ya da daha fazla switch arasında, iki ya da daha fazla kablo ile bağlantı sağlandığında **switch'in iki ya da daha fazla kabloyu tek kablo gibi kullanmasını sağlayan protokoldür**. Diğer üreticiler EtherChannel'a **LAG (Link Aggregation)** da der. Windows server'lerde ise Nic Teaming işlemi benzer bir görevi yerine getirip, sunucunun ethernet portlarını birleştirerek **daha yüksek bant genişlikli** bir erişim sağlar.

Peki bu konfigürasyonlara neden ihtiyaç vardır?

Etherchannel sayesinde hem **yedeklilik (redundancy)** hem **load balance**, hem de **yüksek bandwidth** sağlanmış oluyor. En az iki en fazla 8 bağlantı birleştirilebilir. Etherchannel yapılacak olan portlar aynı hıza sahip olmalı ve aynı vlan'da yer almalıdır. Eğer etherchannel yapılan bağlantılar trunk olacak ise hepsi de trunk olmalıdır.

Etherchannel iki protokol ile yapılır. Bunlardan birincisi Cisco protokolü olan **PAgP** (Port Aggregation Protokolü) ikincisi ise **LACP** (Link Aggregation Control Protocol) IEEE 802.3ad olan standart protokoldür. PAgP iki mode de çalışır, **Desirable** ve **Auto**.

Desirable mode de switch uzaktaki switch'e PAgP paketlerini gönderir ve etherchannel kurmak için davet eder.

Auto mode de ise uzaktaki bir switch PAgP ile etherchannel oluşturmak için istek gönderdiği zaman bağlantıyı kurar.

LACP 3 mode de çalışır, **On**, **Active** ve **Passive**;

Mode on olduğunda interface koşulsuz olarak gider ve etherchannel'e katılır.

Mode passive olduğunda PAgP'deki auto'ya karşılık gelir.

Mode active olduğunda ise PAgP'deki desirable'ye karşılık gelir.

LACP çalışma mantığı port priority değeri en düşük olan portlar etherchannel'e katılır, bunu yapmak için lacp port-priority komutu ile standartta 32768 olan değeri düşürmek gerekir.

Yukarıda etherchannel'in portları arasında yükleri eşit dağıttığından bahsetmiştik. Peki bu işlem nasıl yapılacak biraz da ondan bahsedelim. Frame'ler kullanılan hashing algoritması sonucunda belirli bir porttan forward edilirler. Algoritma source mac address, destination mac address ya da ikisini, source IP address, destination IP address ya da ikisini veya TCP/UDP port numaralarını kullanabilir.

Etherchannel Algoritmaları

```
kat1Switch(config)#port-channel load-balance ?  
dst-ip Dst IP Addr  
dst-mac Dst Mac Addr  
src-dst-ip Src XOR Dst IP Addr  
src-dst-mac Src XOR Dst Mac Addr  
src-ip Src IP Addr  
src-mac Src Mac Addr
```

Listeden de görüldüğü gibi MAC adresi, IP adresi veya port bazında işlemler yapılabilir. Default configuration “src-dst-ip” dir. Eğer layer 2 switch kullanılıyorsa default olarak “src-mac” kullanılır.

Load-balancing metodu seçerken kullanılan yöntemle dikkat etmek gerekir. Mesela, ortamda bir server olduğunu düşünelim; IP’si sabittir, değişmez. Eğer burada load-balancing yöntemi olarak des-ip kullanırsak sürekli aynı link kullanılacak ve diğer linkler kullanılmayacaktır, çünkü load balance değişikliği için **destination** (hedef) IP’nin değişmesi gerekmekte, trafikte yoğun olarak bu aynı IP’ye doğru olduğundan yük kablolar arası dağıtılmamaktadır. Bu da verimli bir çalışma değildir. Böyle bir durumda src-dst-ip kullanmak daha iyi bir çözüm olacaktır. Burada dikkat edilmesi gereken bir nokta var; eğer paketler IP paketi değilse, IPX gibi bir protokol kullanılıyorsa o zaman ne olacak? Tabi ki böyle bir durumda source-destination IP bilgisine bakamaz. O halde diğer load balance metodu yani lowest-method kullanılacaktır.

Etherchannel Komutları

Öncelikle load balance yöntemi belirlenir;

```
Kat1Switch(config)#port-channel load-balance src-dst-ip
```

Sonra etherchannel’e dahil edilecek interface’lerde aşağıdaki komutlar uygulanır;

```
kat1Switch(config-if)#channel-protocol pagp  
kat1Switch(config-if)#channel-group 1 mode desirable
```

Etherchannel Sorun Çözme Komutları

Switch'de kullanılan etherchannel algoritmasını öğrenmek için:

```
kat1Switch#show etherchannel load-balance
```

Hangi portların, hangi mod'da çalıştığını listelemek için:

```
kat1Switch#show etherchannel port-channel
```

Etherchannel bilgilerini özetlemek için;

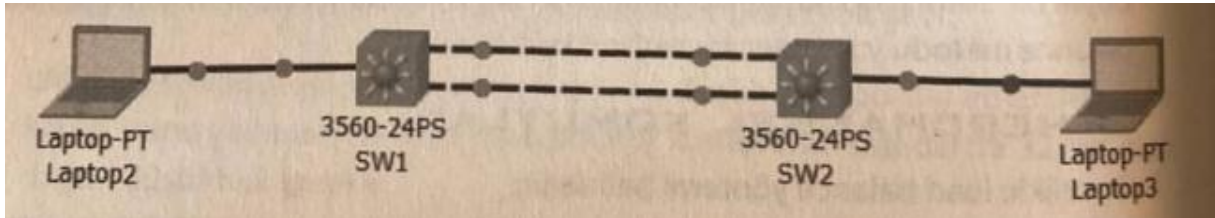
```
kat1Switch#show etherchannel summary
```

LACP konfigürasyonu, PAgP'den farklı olarak system priority bilgisi ister.

```
Switch(config-if)# lacp port-priority [deger]
```

Değer olarak System priority değeri 0 ile 65,535 arasında olabilir, default olarak 32,768'dir. Eğer switch'ler de bu değer ayarlanmazsa MAC adreslerine bakılarak hangisinin rolleri belirleme işini yapacağı belirlenir. En düşük MAC değerine sahip olan seçilecektir.

Etherchannel Örnek Uygulama



Şekildeki gibi bir ağda SW2'de STP etkin olduğundan ve LOOP'u engellemek için bağlantılardan birisi aktif değildir. Biz iki switch arasındaki 2 kabloyu tek bir access mode'da çalışan kablo gibi konfigüre edelim. Normalde iki switch arasındaki bağlantı hızı 1Gbps iken işlem sonrası 2Gbps olacak ve olası kablolardan birisinin kopması durumunda diğeri trafiği aksatmadan devam ettirecektir.

Enter configuration commands, one per line. End with CNTL/Z.

```
Switch(config)#host SW1
SW1(config)#port-channel load-balance src-dst-ip (Source-
Destination-IP yöntemine göre load balance yapılacak)
SW1(config)#
SW1(config)#int ran gi 0/1-2 (şimdi etherchannel daki
interface'lerin ayarlarını yapalım)
SW1(config-if-range)#channel-protocol pagp
SW1(config-if-range)#channel-group 1 mode desirable
SW1(config-if-range)#
Creating a port-channel interface Port-channel 1

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1,
changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1,
changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2,
changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2,
changed state to up

SW1(config-if-range)#
```

Aynı ayarlar diğer switch'de de aynı şekilde yapılır:

```
Switch>en
Switch#conf t
```

Enter configuration commands, one per line. End with CNTL/Z.

```
Switch(config)#host SW2
SW2(config)#port-channel load-balance src-dst-ip
SW2(config)#
SW2(config)#int ran gi 0/1-2
SW2(config-if-range)#channel-protocol pagp
SW2(config-if-range)#channel-group 1 mode desirable
SW2(config-if-range)#
```

Creating a port-channel interface Port-channel 1

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1,
changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1,
changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2,
changed state to down

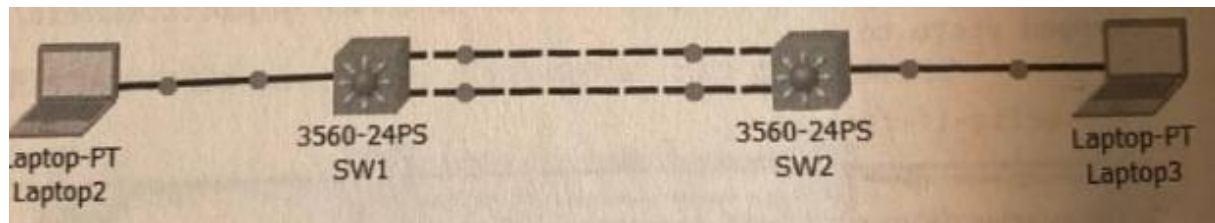
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2,
changed state to up

%LINK-5-CHANGED: Interface Port-channel1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Port-channel1,
changed state to up

SW2(config-if-range)#

İşlemler sonrası iki switch arası bağlantı yedekli bir şekilde devreye girecektir:



Şimdi Bir Uygulama Daha Yapalım

İki L3 switch arasında 3 bağlantı var olsun ve arada trunk bir etherchannel uygulayalım:

```
Switch(config)#host SW1
SW1(config)#interface Por
SW1(config)#interface Port-channel 1
SW1(config-if)#sw tr enc do
SW1(config-if)#sw mode tr
SW1(config-if)#exit
SW1(config)#int ran fa 0/2-4
SW1(config-if-range)#sw tr enc do
SW1(config-if-range)#sw mod tr
SW1(config-if-range)#channel-group 1 mode on
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/2,
changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/2,
changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/3,
changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/3,
changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/4,
changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/4,
changed state to up

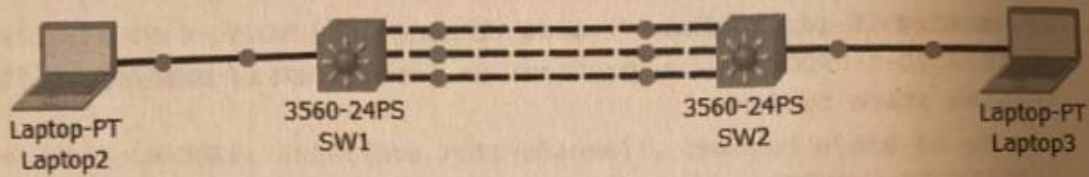
%LINK-5-CHANGED: Interface Port-channell, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Port-channell,
changed state to up

SW1(config-if-range)#

Switch>en
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#host SW2
```

```
SW2(config)#interface port-channel 1
SW2(config-if)#sw tr enc do
SW2(config-if)#sw mod tr
SW2(config-if)#exit
SW2(config)#int ran fa 0/2-4
SW2(config-if-range)#sw tr enc do
SW2(config-if-range)#sw mod tr
SW2(config-if-range)#channel-group 1 mode on
SW2(config-if-range)#
```



Şimdi aradaki etherchannel çalışıyor mu kontrol edelim. Aradaki hat trunk olduğuna göre biz herhangi bir vlan trafiğinin geçip geçmediğini kontrol edeceğiz. Test için her iki switch'de de vlan 10 oluşturup, client'ın bulunduğu portu bu vlan'a üye yapalım:

```
SW1(config)#vlan 10
SW1(config-vlan)#exit
SW1(config)#int fa 0/1
SW1(config-if)#sw mod acc
SW1(config-if)#sw acc vlan 10
SW1(config-if)#exit
SW1(config)#
SW2(config)#vlan 10
SW2(config-vlan)#ex
SW2(config)#
SW2(config)#int fa 0/1
SW2(config-if)#sw mod acc
SW2(config-if)#sw acc vlan 10
SW2(config-if)#exit
şimdi de soldaki bilgisayara 192.168.10.1, diğerine 192.168.10.2
Ip'lerini verip soldan sağa ping atalım:
```

```
Laptop2
Physical Config Desktop Programming Attributes
Command Prompt

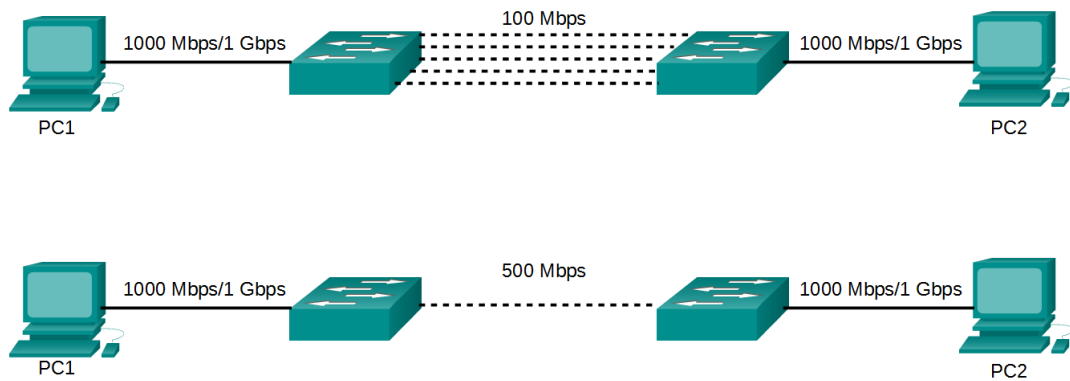
Packet Tracer PC Command Line 1.0
C:\>ping 192.168.10.2

Pinging 192.168.10.2 with 32 bytes of data:

Reply from 192.168.10.2: bytes=32 time=1ms TTL=128
Reply from 192.168.10.2: bytes=32 time<1ms TTL=128
Reply from 192.168.10.2: bytes=32 time<1ms TTL=128
Reply from 192.168.10.2: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.10.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
Approximate round trip times in milli-seconds:
```

Bağlantı Kümeleme (Etherchannel) – Packet Tracer



Switch'ler arası birden fazla bağlantıyı kümeleyerek tek bir port gibi iletişim kurmasına etherchannel denir. Yukarıdaki görsel iki switch arası 5 tane 100 Mbps'lik bağlantı tek bir bağlantı gibi 500 Mbps'lik hale gelmiş. Spanning Tree de bunu tek bir port gibi görür, eğer o şekilde görmeseydi loop algılar ve port kapatırdı. Bu özellik daha pahalı bir switch almaktan bizi kurtarabilir, boşta kalan portları birleştirerek

(kümeleyerek) bağlantıyı hızlandırabilir. Gerçek hayat uygulamasında gigabit portlar kümelenebilir.

Etherchannel faydaları; kanalları birleştirerek bağlantı hızını arttırmak, yedeklilik sağlamak yani port veya kablolardan birinde arıza çıkması durumunda iletişim diğer kanallardan devam eder. Bunun dışında yük dengeleme (load balancing) de yapılabilir.

Tabi dediğimiz gibi etherchannel yapılandırmasından sonra Spanning Tree bunu tek kanal olarak görüyor ve herhangi bir port kapatma söz konusu olmuyor. Protokole göre 8-10 porta kadar kümeleme yapılabilir. Yani 8 tane 100 Mbps'lik portu kümeleyerek 8 portluk bağlantı hızı sağlanabilir.

Etherchannel kurulabilmesi için bazı gereklilikler vardır;

- **Hız ve duplex:** Bağlantı yapılacak iki switch'in port hızlarının uyuşması gerekir, ve aynı duplex modunda (full-duplex) olmalıdır.
- **Mode:** Örneğin bir taraf access mod, diğer taraf trunk modda ise etherchannel kurulamaz. Ya access-access ya da trunk-trunk olması gerekir.
- **Trunk:** Bağlantı yapılan portlarına aynı vlan'a atanmış olması lazım.
- **Trunk Özellikleri:** Eğer trunk yapılandırıldıysa özellikleri de aynı olmalı, mesela allowed durumları, native vlan.

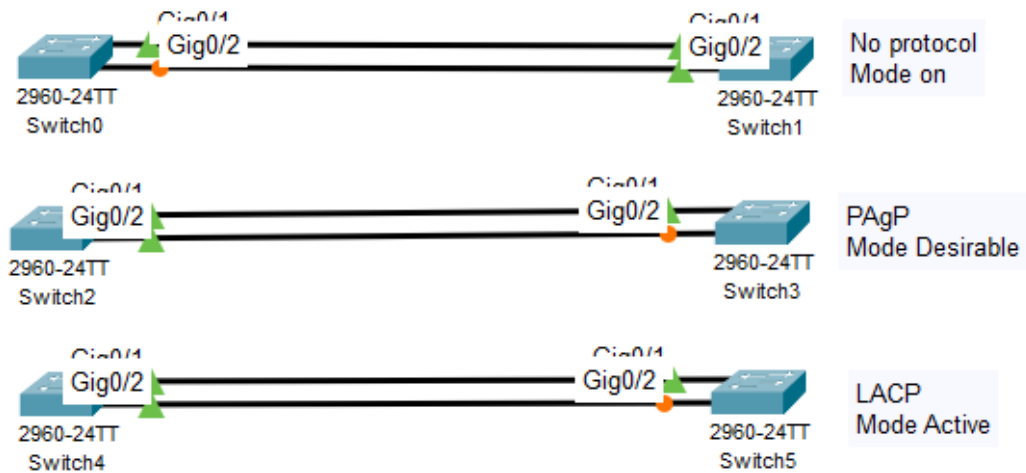
Etherchannel Protokolleri

Etherchannel protokolleri, birden fazla portu tek bir mantıksal bağlantı (Port-Channel) olarak birleştirirken iki switch arasında otomatik anlaşmayı sağlayan yazılım protokolleridir. Bu protokoller sayesinde etherchannel kurulabilir.

1. **PAgP (Port Aggregation Protocol):** Cisco tarafından geliştirilmiştir. Cisco'ya özeldir. Amaç, iki cihaz arasında EtherChannel kurmada önce otomatik olarak uyumluluk ve bağlantı durumunu görüşür. PAgP, karşı tarafa PAgP mesajları gönderir. Bu mesajlarla, portların Hız (Speed), Duplex, Vlan, Trunk ayarları gibi bilgilerini karşılaştırır. İki modu vardır;
 - a. **Desirable:** Aktif olarak PAgP mesajı gönderir ve bağlantı kurmak ister. Auto veya Desirable ile çalışır.
 - b. **Auto:** Pasif olarak PAgP mesajlarını bekler, kendisi başlatmaz. Yalnızca Desirable ile çalışır.
2. **LACP (Link Aggregation Control Protocol):** Cisco, HP, Juniper gibi farklı cihazlarda da çalışır. Amaç, bağlantıların otomatik olarak gruplandırılması ve yük dengeleme işlemlerinin yönetimi. LACP, portlar arasında LACP Data Unit (LACPDU) adı verilen mesajları gönderir. Bu mesajlar sayesinde portların uygunluğu kontrol edilir. Maksimum 16 port tanımlanabilir (8 aktif + 8 yedek). LACP, aktif portlardan biri arızalanırsa otomatik olarak yedek portu devreye alabilir.
 - a. **Active:** Aktif olarak LACP mesajı gönderir ve bağlantı kurmak ister. Active veya Passive ile çalışır.

b. **Passive:** LACP mesajlarını bekler, kendisi başlatmaz. Active ile çalışır.

Aşağıdaki yapıda her biri için etherchannel yapılandırılacaktır:



Switchler gigabit portlarından birbirine bağlanmış durumdadır ve dikkat edersek her birinde portlardan biri spanning tree'den dolayı bloklanmış durumda.

Protokol kullanmadan etherchannel yapılandırma:

Switch0'ın CLI ekranındayız ve ilgili portların (Gig0/1 ve Gig0/2) arayüzüne giriyoruz:

```
Switch(config)#  
Switch(config)#interface range gigabitEthernet 0/1-2  
Switch(config-if-range)#
```

Bir switch'te en fazla 6 grup oluşturabiliyoruz:

```
Switch(config-if-range)#channel-group ?  
<1-6> Channel group number  
Switch(config-if-range)#channel-group |
```

Biz 1'i seçip ilerleyelim ve mode seçimini yapalım (on demek zaten protokolsüz demektir):

```
Switch(config-if-range)#channel-group 1 mode ?  
active      Enable LACP unconditionally  
auto        Enable PAgP only if a PAgP device is detected  
desirable   Enable PAgP unconditionally  
on          Enable Etherchannel only  
passive     Enable LACP only if a LACP device is detected  
Switch(config-if-range)#channel-group 1 mode on  
Switch(config-if-range)#  
Creating a port-channel interface Port-channel 1  
  
%LINK-5-CHANGED: Interface Port-channel1, changed state to up  
  
%LINEPROTO-5-UPDOWN: Line protocol on Interface Port-channel1, changed state to up
```

Aynı işlemi sağdaki switch1 için de yapalım:

```
Switch(config)#interface range gigabitEthernet 0/1-2
Switch(config-if-range)#channel-group 1 mode on
Switch(config-if-range)#
Creating a port-channel interface Port-channel 1

%LINK-5-CHANGED: Interface Port-channel1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Port-channel1, changed state to up
```

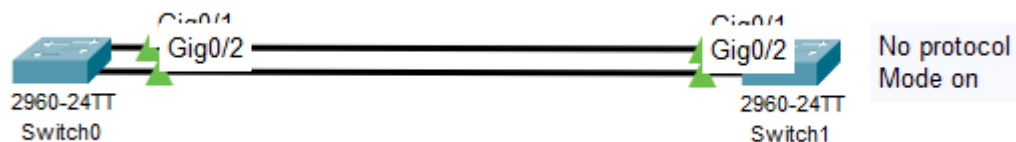
Bakalım etherchannel kurulumunu yapmış mı? Detaylarını görmek için **show etherchannel summary** komutunu kullanabiliriz.

```
Switch#
Switch#show etherchannel summary
Flags:  D - down          P - in port-channel
        I - stand-alone  s - suspended
        H - Hot-standby (LACP only)
        R - Layer3       S - Layer2
        U - in use       f - failed to allocate aggregator
        u - unsuitable for bundling
        w - waiting to be aggregated
        d - default port

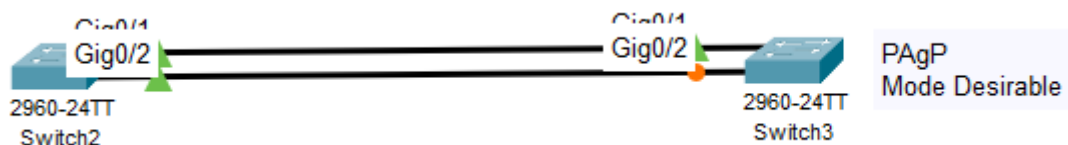
Number of channel-groups in use: 1
Number of aggregators:          1

Group  Port-channel  Protocol    Ports
-----+-----+-----+-----
1      Po1 (SU)      -           Gig0/1 (P) Gig0/2 (P)
Switch#
```

Ve görüldüğü gibi artık bloklanmış bir port yok:



PAgP kullanarak etherchannel yapılandırma:



Switch2 üzerinde;

Bu sefer group 2 dedik. 1 de diyebilirdik zaten switch'ler farklı.

```
Switch(config)#interface range gigabitEthernet 0/1-2
Switch(config-if-range)#channel-group 2 mode desirable
Switch(config-if-range)#
Creating a port-channel interface Port-channel 2

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to down
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2, changed state to down
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2, changed state to up
```

Aynısını sağdaki switch3 için de yapalım:

```
Switch(config)#interface range gigabitEthernet 0/1-2
Switch(config-if-range)#channel-group 2 mode desirable
Switch(config-if-range)#
Creating a port-channel interface Port-channel 2

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to down
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2, changed state to down
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2, changed state to up

%LINK-5-CHANGED: Interface Port-channel2, changed state to up

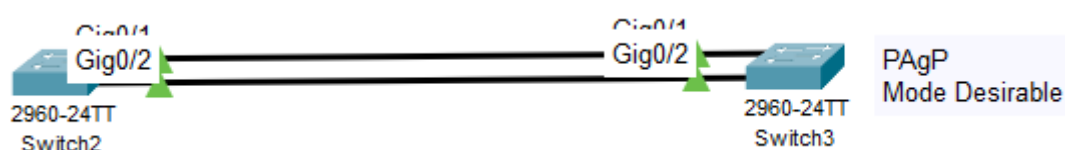
%LINEPROTO-5-UPDOWN: Line protocol on Interface Port-channel2, changed state to up
```

Yapılandırma özetine bakarsak;

```
Switch#show etherchannel summary
Flags:  D - down          P - in port-channel
        I - stand-alone  s - suspended
        H - Hot-standby (LACP only)
        R - Layer3       S - Layer2
        U - in use       f - failed to allocate aggregator
        u - unsuitable for bundling
        w - waiting to be aggregated
        d - default port

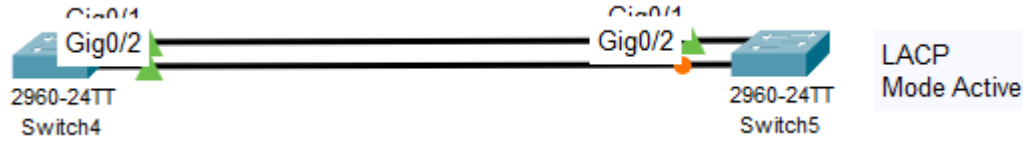
Number of channel-groups in use: 1
Number of aggregators:          1

Group  Port-channel  Protocol    Ports
-----+-----+-----+-----
2      Po2 (SU)        PAgP       Gig0/1 (P) Gig0/2 (P)
Switch#
```



LACP kullanarak etherchannel yapılandırma:

Hatırladığımız üzere LACP marka bağımsızdır, yani diğer cihazlar için de uyumludur. İki taraftaki switch markaları farklı olabilir.



Soldaki switch4 üzerinde:

```
Switch(config)#interface range gigabitEthernet 0/1-2
Switch(config-if-range)#channel-group 3 mode active
Switch(config-if-range)#
Creating a port-channel interface Port-channel 3

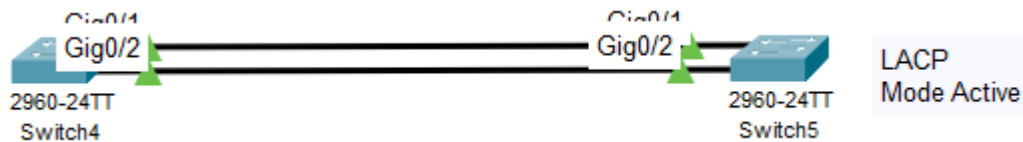
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to down
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2, changed state to down
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2, changed state to up
```

Sağdaki switch5 üzerinde:

```
Switch(config)#interface range gigabitEthernet 0/1-2
Switch(config-if-range)#channel-group 3 mode active
Switch(config-if-range)#
Creating a port-channel interface Port-channel 3

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to down
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2, changed state to down
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2, changed state to up
%LINK-5-CHANGED: Interface Port-channel3, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface Port-channel3, changed state to up
```

Not: karşı taraf pasive modda olsa da olur.

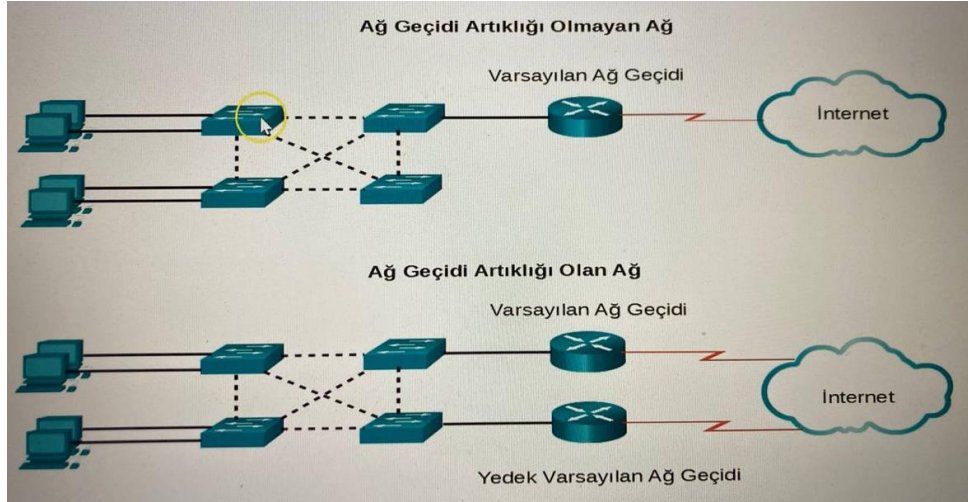


Etherchannel yapılandırmaları bitti. Peki vlan yapılandırmalarını nasıl yapacağız? Tek tek mi? Hayır, oluşturduğumuz port-channel üzerinden yapıyoruz. (Po1)

```
Switch#show interfaces status
Port      Name      Status      Vlan      Duplex  Speed  Type
Po1       Po1       connected   1         auto    auto   
```

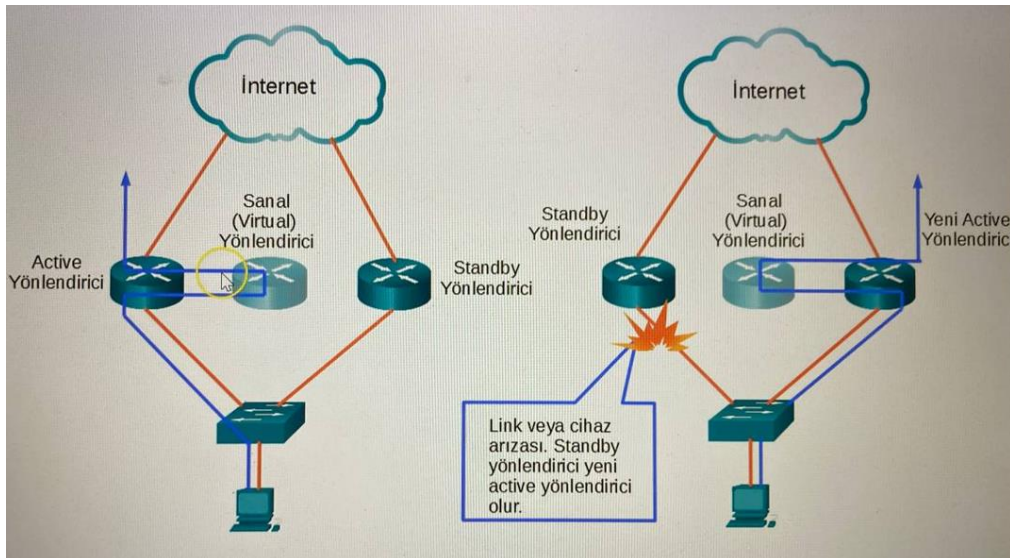
Default Gateway Redundancy

Default Gateway Redundancy, bir ağda **varsayılan ağ geçidinin** (default gateway) devre dışı kalması durumunda kullanıcıların internete veya ağlara erişiminin **kesilmemesi** için kullanılan bir **yedeklilik (redundancy)** mekanizmasıdır.



Sol tarafta iki adet fiziksel yönlendirici (router) var. Bunlar birlikte tek bir sanal (virtual) yönlendirici oluşturur. Bu sanal yönlendiricinin sanal IP adresi, tüm istemcilerin (PC'lerin) default gateway'i oluyor. Soldaki yönlendirici "Active" durumda ve tüm trafik aktif router üzerinden internete çıkıyor. Sağdaki ise; aktif cihazı takip ediyor.

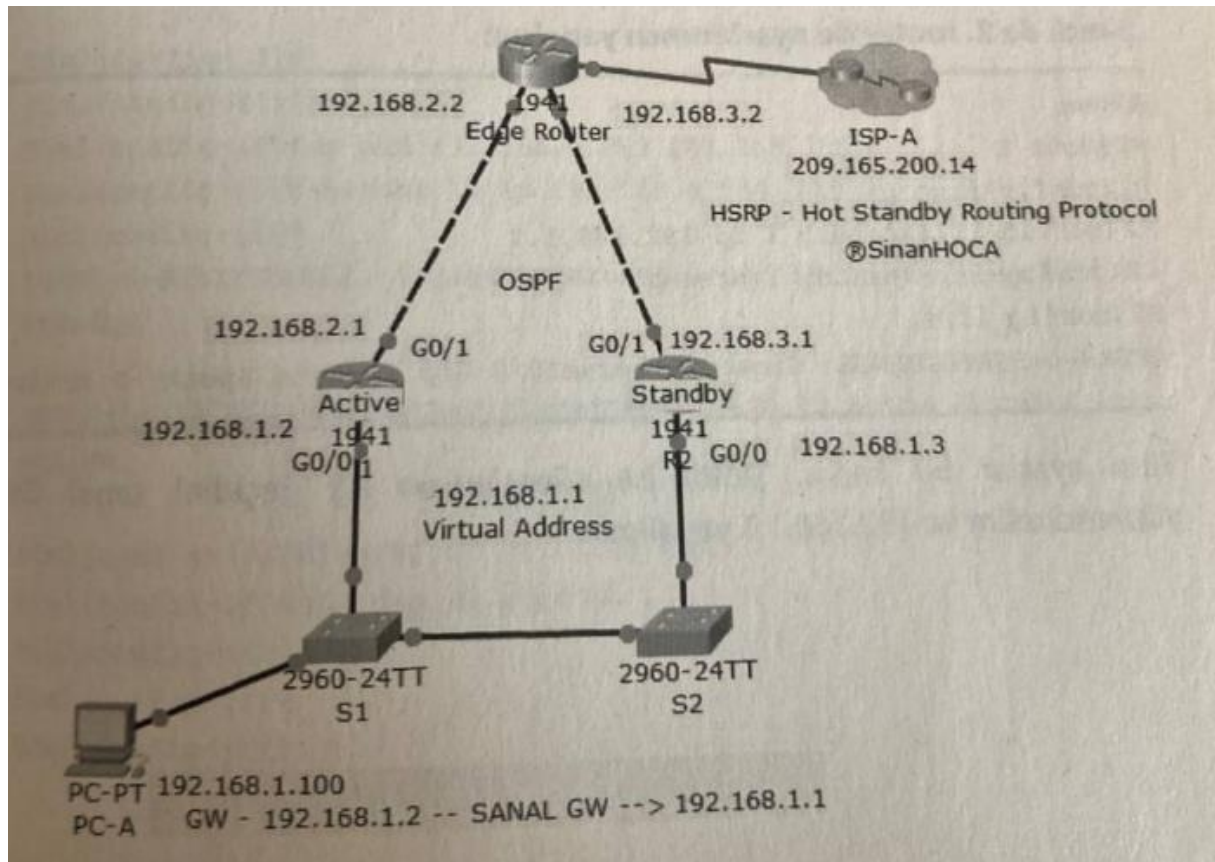
Eğer olur da aktif yönlendirici veya ona giden bağlantı arızalanır ise, standby yönlendirici, hello paketleri kesildiği için bunu tespit eder. Standby router otomatik olarak yeni Active rolünde olur. Bunun sonucunda sanal IP değişmez, bu nedenle PC'ler gateway değiştiğini fark etmezler. Internet bağlantısı da kesintiye uğramadan devam eder.



HSRP Protokolü ve Yapılandırması

HSRP (Hot Standby Router Protocol), Cisco tarafından geliştirilen bir **default gateway yedeklilik** protokolüdür. Böylece sistemdeki router'lardan biri **down** olduğunda tüm sistem çökmez, erişimler devam eder. HSRP, load balance değil **fail over** sağlar. Yani sistemdeki yedekli çalışan router'lerden yalnızca bir tanesi çalışır, yükü router'lar arası dağıtmaz.

HSRP'de router'lar kendi aralarında bir sanal ağ geçidi oluşturur ve hangi router aktif ise o router, bu sanal ağ geçidinin MAC adresi olarak kendi MAC adresini yayınlamaya çalışır, trafiği üzerine alır.



Şimdi konfigürasyonu yapalım. Konfigürasyonu router'ların client'lere bakan interface'sinin altında yapıyoruz:

```
R1#conf t
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)#int gi 0/0
R1(config-if)#standby 1 ip 192.168.1.1
R1(config-if)#standby 1 priority 105
```

Burada biz R1'e birincil router'sın demiş olduk. Çünkü normalde öncelik numarası varsayılan olarak 100'dür. Bu değerden daha büyük değerler verildiğinde önceliği artırılmış olur.

```
%HSRP-6-STATECHANGE: GigabitEthernet0/0 Grp 1 state Speak -> Standby
%HSRP-6-STATECHANGE: GigabitEthernet0/0 Grp 1 state Standby ->
Active
```

```
R1(config-if)#standby 1 preempt
R1(config-if)#standby 1 track g 0/1
```

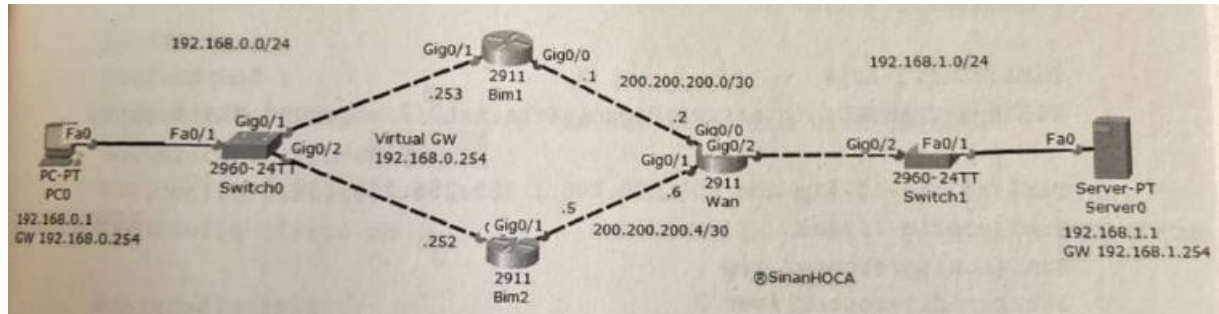
Bu komut ile takip edeceğimiz dış interface'miz belirlenmiş olur. Bu interface'in kopması durumunda öncelik numarası 10 azalır ve bizim verdiğimiz değerden sonra 95'e düşer. Diğer router'ın varsayılan değeri 100 olduğundan devreye girecektir.

Şimdi de 2. Router'da ayarlamalarımızı yapalım:

```
R2>en
R2#conf t
R2(config)#int gi 0/0
R2(config-if)#standby 1 ip 192.168.1.1
R2(config-if)#standby 1 preempt
R2(config-if)#
%HSRP-6-STATECHANGE: GigabitEthernet0/0 Grp 1 state Speak -> Active
```

Tüm ayarlar bu kadar. Şimdi ise client'ımızın ağ geçidini sanal gateway'e yönlendirelim ve **192.168.1.1** yapalım.

Şimdi bir örnek daha yapalım:



```
Router>en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#host Biml
Biml(config)#int gi 0/1
Biml(config-if)#no sh

Biml(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1,
changed state to up

Biml(config-if)#
Biml(config-if)#
Biml(config-if)#ip add 192.168.0.253 255.255.255.0
Biml(config-if)#standby 10 ip 192.168.0.254
Biml(config-if)#
%HSRP-6-STATECHANGE: GigabitEthernet0/1 Grp 10 state Speak ->
Standby

%HSRP-6-STATECHANGE: GigabitEthernet0/1 Grp 10 state Standby ->
Active

Biml(config-if)#standby 10 priority 105
Biml(config-if)#standby 10 preempt
Biml(config-if)#
Biml(config-if)#
Biml(config-if)#
```

```
Bim1(config-if)#int gi 0/0
Bim1(config-if)#no sh
```

```
Bim1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up
```

```
Bim1(config-if)#ip add 200.200.200.1 255.255.255.252
```

```
Bim1(config-if)#ex
```

```
Bim1(config)#router rip
```

```
Bim1(config-router)#ver 2
```

```
Bim1(config-router)#net 192.168.0.0
```

```
Bim1(config-router)#net 200.200.200.0
```

```
Bim1(config-router)#
```

```
Bim1(config-router)#ex
```

```
Bim1(config)#
```

```
Router>en
```

```
Router#conf t
```

```
Enter configuration commands, one per line. End with CNTL/Z.
```

```
Router(config)#host Bim2
```

```
Bim2(config-if)#int gi 0/2
```

```
Bim2(config-if)#no sh
```

```
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2,
changed state to up
```

```
Bim2(config-if)#
```

```
Bim2(config-if)#ip add 192.168.0.252 255.255.255.0
```

```
Bim2(config-if)#standby 10 ip 192.168.0.254
```

```
Bim2(config-if)#
```

```
%HSRP-6-STATECHANGE: GigabitEthernet0/2 Grp 10 state Speak ->
Standby
```

```
Bim2(config-if)#int gi 0/1
```

```
Bim2(config-if)#no sh
```

```
Bim2(config-if)#ip add 200.200.200.5 255.255.255.252
```

```
Bim2(config-if)#ex
```

```
Bim2(config)#router rip
```

```
Bim2(config-router)#ver 2
```

```
Bim2(config-router)#net 192.168.0.0
```

```
Bim2(config-router)#net 200.200.200.4
```

```
Bim2(config-router)#
```

```
Router>en
```

```
Router#conf t
```

```
Enter configuration commands, one per line. End with CNTL/Z.
```

```
Router(config)#host Wan
```

```
Wan(config)#int gi 0/2
```

```
Wan(config-if)#no sh
```

```
Wan(config-if)#
```

```
%LINK-5-CHANGED: Interface GigabitEthernet0/2, changed state to up
```

```
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/2,  
changed state to up
```

```
Wan(config-if)#ip add 192.168.1.254 255.255.255.0
```

```
Wan(config-if)#int gi 0/0
```

```
Wan(config-if)#no sh
```

```
Wan(config-if)#
```

```
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up
```

```
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0,  
changed state to up
```

```
Wan(config-if)#ip add 200.200.200.2 255.255.255.252
```

```
Wan(config-if)#ex
```

```
Wan(config)#int gi 0/1
```

```
Wan(config-if)#no sh
```

```
Wan(config-if)#
```

```
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up
```

```
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1,  
changed state to up
```

```
Wan(config-if)#
```

```
Wan(config-if)#ip add 200.200.200.6 255.255.255.252
```

```
Wan(config-if)#
```

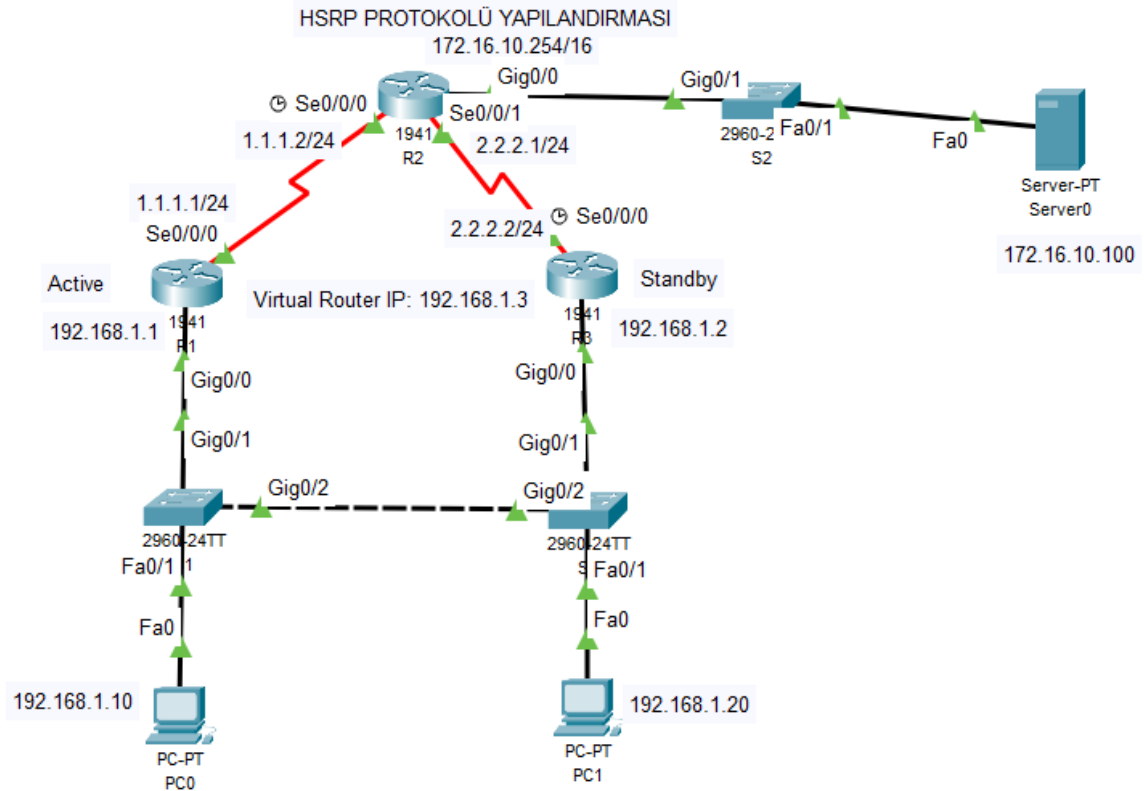
```
Wan(config-if)#ex
```

```
Wan(config)#router rip
Wan(config-router)#ver 2
Wan(config-router)#net 192.168.1.0
Wan(config-router)#net 200.200.200.0
Wan(config-router)#net 200.200.200.4
Wan(config-router)#exit
Wan(config)#
```

HSRP Durum Kontrolü: show standby

Packet Tracer Uygulaması

Aşağıdaki gibi bir yapımız olsun:



Bu yapıda şuan routerlar için **RIPv2 yönlendirmesi** yapılandırılmıştır.

HSRP Version 1 ve Version 2, Cisco'nun default gateway yedekliliğini sağlamak için geliştirdiği aynı protokolün iki farklı sürümüdür. Her iki sürüm de Active-Standby yapısı ile çalışır ancak teknik anlamda bazı önemli farklara sahiptir.

Version 1 eski sürümdür ve **multicast** iletişimde **224.0.0.2** adresini kullanır. Version 2 ise daha güncel bir sürüm olup sadece HSRP için ayrılmış olan **224.0.0.102** multicast adresini kullanır; bu sayede daha izole ve optimize edilmiş bir haberleşme sağlar. Ayrıca HSRP Version 1 yalnızca **0 ile 255** arasında grup numarası desteklerken, Version 2 bu sınırı **4096'ya** kadar çıkarır. Bu da özellikle çok fazla VLAN kullanılan büyük kurumsal ağlarda Version 2'nin tercih edilmesini sağlar.

Bunun yanında HSRP Version 1 IPv6'yı desteklemezken, Version 2 hem IPv4 hem de IPv6 ortamlarında çalışabilir. Sanal MAC adresi yapısı da iki sürümde farklıdır. Version 1'de MAC adresi şu formattadır; **0000.0C07.ACxx** şeklindedir (xx → HSRP group number (0-255)). Version 2 de ise şu formattadır; **0000.0C9F.Fxxx** şeklinde (xxx → HSRP group number (0-4095)).

O zaman yapılandırmamıza R1 üzerinden başlayarak gerçekleştirelim. R1 üzerinden local area network'e bakan interface'in arayüzüne (Gig0/0) giriyoruz:

Grup numarası ve sanal IP adresini belirledik.

```
R1(config)#int gigabitEthernet 0/0
R1(config-if)#standby ?
<0-4095>  group number
ip        Enable HSRP and set the virtual IP address
ipv6      Enable HSRP IPv6
preempt   Overthrow lower priority Active routers
priority  Priority level
timers    Hello and hold timers
track     Priority Tracking
version   HSRP version
R1(config-if)#standby 10 ip 192.168.1.3
R1(config-if)#
%HSRP-6-STATECHANGE: GigabitEthernet0/0 Grp 10 state Speak -> Standby
%HSRP-6-STATECHANGE: GigabitEthernet0/0 Grp 10 state Standby -> Active
```

Priority değerini 0-255 arasından 150 olarak verdik. Kimin priority değeri yüksek ise o Aktif router rolünü üstlenir.

```
R1(config-if)#standby 10 ?
ip        Enable HSRP and set the virtual IP address
ipv6      Enable HSRP IPv6
preempt   Overthrow lower priority Active routers
priority  Priority level
timers    Hello and hold timers
track     Priority Tracking
R1(config-if)#standby 10 priority ?
<0-255>   Priority value
R1(config-if)#standby 10 priority 150
R1(config-if)#
```

Preempt'in anlamı ise şu; bizim senaryomuzda aktif router bozulduğunda standby durumunda olan router trafik iletim rolünü üstleniyor. Eğer bizim bozulan

router'ımızda sorun giderilirse tekrar aktifliğin kimin üzerinde olacağını bu komut ile belirliyoruz. Yani preempt şeklinde belirlemez isek, yeni aktif olarak router aktif olarak trafiği iletmeye devam eder. Eğer preempt şeklinde ifade edersek, başlangıçta aktif olarak router görevini geri üstlenir.

```
R1(config-if)#standby 10 pre
R1(config-if)#standby 10 preempt
R1(config-if)#
```

Timer gibi diğer parametreler ccnp'de anlatılan konulardır.

Şimdi sağdaki (R3) üzerinde yapılandırmamızı yapmaya devam edelim:

```
R3(config-if)#interface gigabitEthernet 0/0
R3(config-if)#standby ?
<0-4095> group number
ip Enable HSRP and set the virtual IP address
ipv6 Enable HSRP IPv6
preempt Overthrow lower priority Active routers
priority Priority level
timers Hello and hold timers
track Priority Tracking
version HSRP version
R3(config-if)#standby 10 ip 192.168.1.3
R3(config-if)#
%HSRP-6-STATECHANGE: GigabitEthernet0/0 Grp 10 state Speak -> Standby
```

Router'ımızı standby durumuna getirdik. İstersek priority değerini de verebiliriz, zaten default durumda 100 olduğunu söylemiştik.

Tekrar R1 üzerinden yaptığımız standby yapılandırmasını **show standby** komutu ile görelim:

```
R1#
R1#show standby
GigabitEthernet0/0 - Group 10
  State is Active
    21 state changes, last state change 02:20:40
  Virtual IP address is 192.168.1.3
  Active virtual MAC address is 0000.0C07.AC0A
    Local virtual MAC address is 0000.0C07.AC0A (v1 default)
  Hello time 3 sec, hold time 10 sec
    Next hello sent in 2.257 secs
  Preemption enabled
  Active router is local
  Standby router is 192.168.1.2
  Priority 150 (configured 150)
  Group name is hsrp-Gig0/0-10 (default)
R1#
```

Yukarıda belirttiği MAC adresi, her iki router'ın da bir ARP isteğinde döndüreceği MAC adresidir. Ayrıca 3 saniyede bir Hello Paketleri gönderdiğini, ve 10 saniye hold

time süresi olduğunu yani eğer 10 saniye içinde bir hello mesajı gelmez ise aktiflik durumunun diğer router'a geçeceğini belirtiyor.

Aşağıdaki gibi bir özet bilgi de alabiliriz:

```
R1#show standby brief
                P indicates configured to preempt.
                |
Interface      Grp  Pri P State      Active      Standby      Virtual IP
Gig0/0         10   150 P Active     local       192.168.1.2  192.168.1.3
R1#
```

Aynı bilgilere R3 için de bakarsak:

```
R3#show standby
GigabitEthernet0/0 - Group 10
  State is Standby
    3 state changes, last state change 02:05:00
  Virtual IP address is 192.168.1.3
  Active virtual MAC address is 0000.0C07.AC0A
  Local virtual MAC address is 0000.0C07.AC0A (v1 default)
  Hello time 3 sec, hold time 10 sec
    Next hello sent in 1.44 secs
  Preemption disabled
  Active router is 192.168.1.1
  Standby router is local
  Priority 100 (default 100)
  Group name is hsrp-Gig0/0-10 (default)
R3#
```

```
R3#show standby brief
                P indicates configured to preempt.
                |
Interface      Grp  Pri P State      Active      Standby      Virtual IP
Gig0/0         10   100 Standby     192.168.1.1 local       192.168.1.3
R3#
```

Bu arada PC'lerin default gateway adresleri 192.168.1.3'ü gösteriyor. Şimdi PC0 için arp tablosuna bakalım (sunucuya ping attıktan sonra):

```
C:\>arp -a
Internet Address      Physical Address      Type
192.168.1.1           00e0.f7b5.7b01       dynamic
192.168.1.2           0004.9a08.bd01       dynamic
192.168.1.3           0000.0c07.ac0a       dynamic
C:\>
```

Peki bakalım gönderilen paket nerelerden gidiyor:

PC0 için: (Aslında bence burda 2. Hop 1.1.1.2 olmalıydı)

```
C:\>tracert 172.16.10.100

Tracing route to 172.16.10.100 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      192.168.1.1
  2    9 ms      0 ms      1 ms      2.2.2.1
  3    7 ms      0 ms      1 ms      172.16.10.100

Trace complete.

C:\>|
```

PC1 için:

```
C:\>tracert 172.16.10.100

Tracing route to 172.16.10.100 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      192.168.1.1
  2    0 ms      3 ms      0 ms      2.2.2.1
  3    1 ms      1 ms      0 ms      172.16.10.100

Trace complete.

C:\>|
```

Tamam o zaman şimdi arızayı simüle edelim:

PC0 üzerinden sunucu tarafına sürekli bir istek olsun:

```
C:\>ping -t 172.16.10.100

Pinging 172.16.10.100 with 32 bytes of data:

Reply from 172.16.10.100: bytes=32 time=2ms TTL=126
Reply from 172.16.10.100: bytes=32 time=2ms TTL=126
Reply from 172.16.10.100: bytes=32 time=1ms TTL=126
|
```

Şimdi R1 üzerinden Gig0/0 portunu kapatalım:

R1

GLOBAL

Settings

Algorithm Settings

ROUTING

Static

RIP

SWITCHING

VLAN Database

INTERFACE

GigabitEthernet0/0

GigabitEthernet0/1

Serial0/0/0

Serial0/0/1

GigabitEthernet0/0

Port Status

Bandwidth

Duplex

MAC Address

IP Configuration

IPv4 Address

Subnet Mask

Tx Ring Limit

1000 Mbps

100 Mbps

10 Mbps

Auto

Half Duplex

Full Duplex

Auto

00E0.F7B5.7B01

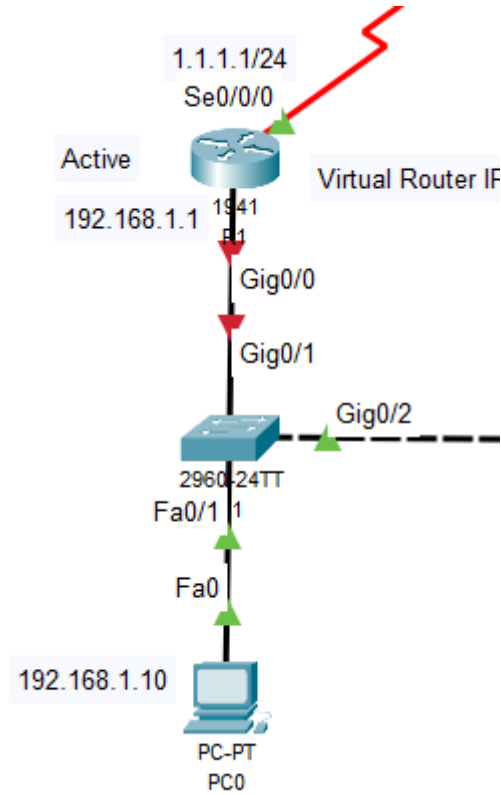
192.168.1.1

255.255.255.0

10

Equivalent IOS Commands

```
R1(config)#interface GigabitEthernet0/0
R1(config-if)#shutdown
R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to administratively down
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to down
%HSRP-6-STATECHANGE: GigabitEthernet0/0 Grp 10 state Active -> Init
```



Gönderilen paketlerin arada bir kesintiye uğradığını fakat gönderilmeye devam edildiğini görebiliriz:

```
Reply from 172.16.10.100: bytes=32 time=13ms TTL=126
Reply from 172.16.10.100: bytes=32 time=14ms TTL=126
Request timed out.
Request timed out.
Reply from 172.16.10.100: bytes=32 time=1ms TTL=126
Reply from 172.16.10.100: bytes=32 time=32ms TTL=126
```

Tekrar tracert ile yolu kontrol ettiğimizde R3 üzerinden gittiğini görebiliriz:

```
C:\>tracert 172.16.10.100

Tracing route to 172.16.10.100 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      192.168.1.2
  2    3 ms      1 ms      0 ms      2.2.2.1
  3    1 ms      0 ms      0 ms      172.16.10.100

Trace complete.

C:\>|
```

Anlık durumda standby bilgilerini almak istersek:

```
R1#show standby brief
                P indicates configured to preempt.
                |
Interface    Grp  Pri  P State    Active        Standby        Virtual IP
Gig0/0       10   150  P Init     unknown       unknown        192.168.1.3
R1#
```

```
R3#show standby brief
                P indicates configured to preempt.
                |
Interface    Grp  Pri  P State    Active        Standby        Virtual IP
Gig0/0       10   100  Active local     unknown       192.168.1.3
R3#
```

R1'deki arıza düzeltilirse tekrar Active konumuna geçecektir çünkü preemption özelliği açık.